Package 'snowflakeR'

May 22, 2026

Title R Interface to the Snowflake ML Platform

Version 0.2.0

Author Simon Field [aut, cre], Snowflake Inc. [cph]

Maintainer Simon Field <simon.field@snowflake.com>

Description Provides native R functions for working with the Snowflake ML platform, including Model Registry (log, deploy, and serve R models), Feature Store (create and manage feature views using dplyr/dbplyr or SQL), experiment tracking, model monitoring, and REST inference to SPCS model serving endpoints. Includes optional parallel and distributed R workflows ('foreach' via doSnowflake, SPCS job execution, and 'crew' integration). Uses 'reticulate' to bridge to the 'snowflake-ml-python' SDK for ML operations. Standard database connectivity (DBI, dbplyr) is provided by the 'RSnowflake' package. Supports both Snowflake Workspace Notebooks and local R environments.

License Apache License 2.0 | file LICENSE

URL https://github.com/Snowflake-Labs/snowflakeR

BugReports https://github.com/Snowflake-Labs/snowflakeR/issues

Encoding UTF-8

Collate 'snowflakeR-package.R'
'python_deps.R'
'connect.R'
'helpers.R'
'version.R'
'query.R'
'dbi.R'
'admin.R'
'datasets.R'
'dosnowflake.R'
'dosnowflake_queue.R'
'dosnowflake_serialize.R'
'dosnowflake_setup.R'
'dosnowflake_tasks.R'
'many_model.R'
'features.R'
'features_online.R'
'registry.R'
'monitoring.R'

'experiments.R'
'rest_inference.R'
'workspace.R'
'crew_spcs.R'
'executor.R'
'r_executor.R'
'zzz.R'

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.1.0)

Imports methods,
reticulate (>= 1.34),
cli,
rlang,
jsonlite

Suggests RSnowflake,
DBI,
snowflakeauth,
RcppTOML,
httr2 (>= 1.0.0),
testthat (>= 3.0.0),
foreach,
iterators,
uuid,
dplyr,
dbplyr,
arrow,
knitr,
parsnip,
rmarkdown,
tune,
withr,
workflows,
yaml

Config/testthat/edition 3

VignetteBuilder knitr

SystemRequirements Python (>= 3.9), snowflake-ml-python (>= 1.5.0)

R topics documented:

`crew_controller_spcs` *Create a crew Controller with SPCS Workers*

Description

Convenience function that creates a `crew` controller configured to launch workers on Snowflake SPCS. Workers are launched as ephemeral EXECUTE JOB SERVICE instances that connect back to the controller via TCP.

Usage

```
crew_controller_spcs(
  conn,
  compute_pool,
  image,
  workers = 4L,
  seconds_idle = 120,
  host = "auto",
  port = 5555L,
  instance_family = "CPU_X64_S",
  eai = ""
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`compute_pool`	Character. SPCS compute pool for workers.
`image`	Character. Docker image URI with R + crew + mirai pre-installed.
`workers`	Integer. Maximum number of concurrent workers (default 4).
`seconds_idle`	Numeric. Seconds before idle workers are terminated (default 120).

host	Character. Controller hostname. If "auto", determines the correct host based on the execution environment.
port	Integer. Controller listener port (default 5555).
instance_family	Character. SPCS instance family (default "CPU_X64_S").
eai	Character. External Access Integration name.

Value

A crew_controller object.

Examples

```
## Not run:
library(crew)
library(snowflakeR)

conn <- sfr_connect()
controller <- crew_controller_spcs(
  conn, compute_pool = "MY_POOL",
  image = "/DB/SCHEMA/REPO/r_crew:latest",
  workers = 8
)
controller$start()

# Submit tasks
controller$push(quote(1 + 1))
controller$pop()

controller$terminate()

## End(Not run)
```

---

crew_launcher_spcs *crew Launcher for SPCS*

Description

R6 class that implements the crew launcher interface for Snowflake SPCS. Each launch_worker() call creates an EXECUTE JOB SERVICE that runs mirai::daemon() connecting back to the controller.

Usage

```
crew_launcher_spcs
```

Format

An object of class NULL of length 0.

print.sfr_connection *Print an sfr_connection object*

Description

Print an sfr_connection object

Usage

```
## S3 method for class 'sfr_connection'
print(x, ...)
```

Arguments

x	An sfr_connection object.
...	Ignored.

Value

Invisibly returns x.

rcat *Print text cleanly*

Description

A cat()-like function that avoids Workspace Notebook formatting issues.

Usage

```
rcat(...)
```

Arguments

...	Objects to concatenate and print.

Value

Invisibly returns NULL.

`registerDoSnowflake` *Register the doSnowflake parallel backend*

Description

Registers a `foreach` parallel backend that dispatches iterations to Snowflake compute.

Usage

```
registerDoSnowflake(
  conn,
  mode = c("local", "tasks", "spcs", "queue"),
  workers = "auto",
  ...
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`mode`	Character. • `"local"` – socket cluster in the current container (default). • `"tasks"` – Snowflake Task graph + SPCS jobs. • `"queue"` – Hybrid Table queue with SPCS workers (ephemeral or persistent). • `"spcs"` – stage-based SPCS job dispatch (not yet implemented).
`workers`	Integer or `"auto"`. Number of parallel workers. `"auto"` (default) detects available CPU cores and reserves one for the main thread.
`...`	Backend-specific options. For `mode = "tasks"`: • `compute_pool` (required) – name of the SPCS compute pool. • `image_uri` (required) – worker Docker image URI in image repo. • `warehouse` – optional Snowflake warehouse for the Task DAG root task. When omitted, Snowflake may create a **serverless** task graph, which requires the `EXECUTE MANAGED TASK` account privilege on the session role. Set this (for example to the same warehouse as your lab `context`) so the graph is **user-managed** and runs without that privilege. • `stage` – stage name (default `"DOSNOWFLAKE_STAGE"`). • `timeout_min` – max minutes to wait for completion (default 30). • `poll_sec` – seconds between status polls (default 5). • `chunks_per_job` – number of SPCS jobs to create (default `"auto"`). • `result_sync_wait_sec` – max seconds to poll `LIST` on the stage before `GET`ing chunk results (default `45`). SPCS volume writes can lag behind Task graph `SUCCEEDED`; increase if you see error 253006. • `result_sync_poll_sec` – seconds between `LIST` polls (default `3`). For `mode = "queue"`: • `compute_pool` (required for ephemeral) – name of the SPCS compute pool. • `image_uri` (required for ephemeral) – worker Docker image URI.

- worker_type – "ephemeral" (default) or "persistent".
- n_workers – number of worker instances (default 4).
- queue_fqn – Hybrid Table name (default "CONFIG.DOSNOWFLAKE_QUEUE").
- stage – stage name (default "DOSNOWFLAKE_STAGE").
- timeout_min – max minutes to wait (default 30).
- poll_sec – seconds between status polls (default 5).
- stale_timeout_sec – seconds before re-queuing stale items (default 600).
- chunks_per_job – number of chunks to create (default "auto").
- pre_warm – logical. Wait for all workers to be READY before enqueuing work (default FALSE). Useful for benchmarking or when consistent timing is important. For smaller jobs, leaving this FALSE lets early workers start immediately.
- instance_family – character. SPCS instance family for resource sizing (default "CPU_X64_S"). Resources are auto-sized: XS=1c/4G, S=3c/12G, M=6c/24G, L=12c/48G, XL=24c/96G.
- result_sync_wait_sec, result_sync_poll_sec – same as for mode = "tasks" (stage result collection after workers finish).

Details

Local (default): iterations run on a cached parallel::makeCluster() socket cluster in the current R session (Workspace container or local). No SPCS objects are required. Call stopDoSnowflake() to shut down the cluster explicitly.

Tasks: iterations are chunked and executed via Snowflake Tasks and SPCS jobs. Requires a compute pool, worker image, and stage; see sfr_dosnowflake_setup() and the mode = "tasks" arguments above.

Queue: iterations are written to a Hybrid Table queue and processed by SPCS workers (ephemeral or persistent). Requires queue DDL, images, and pool configuration; see sfr_dosnowflake_setup() and the shipped notebooks under inst/notebooks/ (workspace_parallel_spcs_*.ipynb, snowflaker_parallel_sp

Spcs: reserved for future stage-only job dispatch; not implemented (an error is raised if selected).

Related infrastructure: sfr_dosnowflake_build_image() for worker images; crew_controller_spcs() and sfr_execute_r_script() for other SPCS parallel patterns beyond foreach.

Value

Invisibly returns TRUE.

Examples

```
## Not run:
library(foreach)
conn <- sfr_connect()
registerDoSnowflake(conn)

result <- foreach(i = 1:10, .combine = c) %dopar% {
  i^2
}

## End(Not run)
```

rglimpse *Glimpse a data.frame*

Description

Shows structure information similar to `dplyr::glimpse()` but without requiring dplyr.

Usage

```
rglimpse(x)
```

Arguments

`x`	A data.frame.

Value

Invisibly returns `x`.

rprint *Print a data.frame cleanly*

Description

Bypasses extra formatting that can cause issues in Workspace Notebooks. Sets a wide print width to prevent line wrapping in notebook cells.

Usage

```
rprint(x, width = 200L, ...)
```

Arguments

`x`	A data.frame or other printable object.
`width`	Integer. Print width in characters. Default: 200.
`...`	Additional arguments passed to `print()`.

Value

Invisibly returns `x`.

rview *View the head of a data.frame*

Description

View the head of a data.frame

Usage

```
rview(x, n = 10L, width = 200L)
```

Arguments

x	A data.frame.
n	Integer. Number of rows to show. Default: 10.
width	Integer. Print width in characters. Default: 200.

Value

Invisibly returns the first n rows.

sfr_add_monitor *Add a model monitor*

Description

Registers a model monitor in the Snowflake Model Registry using the Python SDK. Requires snowflake-ml-python $\geq$ 1.7.1.

Usage

```
sfr_add_monitor(reg, monitor_name, source_config, monitor_config)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
monitor_name	Character. Name for the monitor.
source_config	An object from sfr_monitor_source().
monitor_config	An object from sfr_monitor_config().

Value

Invisibly TRUE on success.

Examples

```
## Not run:
reg <- sfr_model_registry(conn, "ML_DB", "MODELS")
src <- sfr_monitor_source("ML_DB.MY_SCH.LOG", "TS", "SCORE")
cfg <- sfr_monitor_config("M", "v1", warehouse = "ML_WH")
sfr_add_monitor(reg, "MON1", src, cfg)

## End(Not run)
```

`sfr_add_monitor_segment` *Add a segment column to a model monitor*

Description

Add a segment column to a model monitor

Usage

```
sfr_add_monitor_segment(conn, monitor_name, column)
```

Arguments

`conn`	An `sfr_connection` object.
`monitor_name`	Character.
`column`	Character. Segment column name (string column in source data).

Value

Invisibly `TRUE`.

Examples

```
## Not run:
sfr_add_monitor_segment(conn, "MY_MON", "REGION")

## End(Not run)
```

sfr_add_workers *Add workers to a running queue job*

Description

Launches additional ephemeral SPCS workers that claim from the same queue. Use this to speed up a long-running job mid-flight.

Usage

```
sfr_add_workers(
  conn,
  job_id,
  n,
  compute_pool,
  image_uri,
  queue_fqn = "CONFIG.DOSNOWFLAKE_QUEUE",
  instance_family = "CPU_X64_S"
)
```

Arguments

conn	An sfr_connection object.
job_id	Character. Job ID to work on.
n	Integer. Number of additional workers.
compute_pool	Character. Compute pool name.
image_uri	Character. Worker Docker image URI.
queue_fqn	Character. Queue table name.
instance_family	Character. Instance family for resource sizing.

Value

Invisibly returns the number of workers added.

sfr_attach_feature_desc *Attach per-feature descriptions*

Description

Attach per-feature descriptions

Usage

```
sfr_attach_feature_desc(fs, name, version, descs)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.
descs	A named character vector or list mapping feature names to descriptions.

Value

Invisibly returns TRUE.

---

sfr_benchmark_inference

Benchmark SPCS model inference

Description

Runs n inference requests against a deployed SPCS service and reports timing statistics. Use this to validate that the service is responding correctly and to measure throughput.

Usage

```
sfr_benchmark_inference(
  reg,
  model_name,
  new_data,
  service_name,
  n = 10L,
  version_name = NULL,
  verbose = TRUE
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Registered model name.
new_data	A data.frame with input data.
service_name	Character. SPCS service name.
n	Integer. Number of inference iterations. Default: 10.
version_name	Character. Model version (default: model's default).
verbose	Logical. Print per-iteration results? Default: TRUE.

Value

A list (invisibly) with:

- timings: numeric vector of per-iteration seconds
- mean_sec: mean latency
- median_sec: median latency

- `min_sec`, `max_sec`: range
- `total_sec`: total wall time
- `n`: iterations completed
- `last_result`: data.frame from the last prediction

Examples

```
## Not run:
bench <- sfr_benchmark_inference(
  reg, "MY_MODEL", test_data,
  service_name = "my_svc", n = 20
)

## End(Not run)
```

`sfr_benchmark_rest` *Benchmark SPCS inference via REST*

Description

Runs `n` inference requests directly via the REST endpoint and reports timing statistics. Measures true service latency without Python bridge overhead.

Usage

```
sfr_benchmark_rest(
  endpoint,
  new_data,
  n = 10L,
  token = NULL,
  conn = NULL,
  method = "predict",
  verbose = TRUE
)
```

Arguments

`endpoint`	An `sfr_endpoint` object or URL string.
`new_data`	A data.frame with input data.
`n`	Integer. Number of iterations. Default: 10.
`token`	Character or NULL. Snowflake PAT.
`conn`	An `sfr_connection` object, or NULL. Used for auto-generating a PAT when `token` is NULL.
`method`	Character. Model method name. Default: `"predict"`.
`verbose`	Logical. Print per-iteration results? Default: `TRUE`.

Value

A list (invisibly) with timing stats (same structure as `sfr_benchmark_inference()`).

Examples

```
## Not run:
ep <- sfr_service_endpoint(conn, "mpg_service")
bench <- sfr_benchmark_rest(ep, new_data, n = 20)

## End(Not run)
```

`sfr_build_model_index` *Build a Model Index from a Directory Table View*

Description

Queries the MODEL_INDEX view (or any view with PARTITION_KEY and RELATIVE_PATH columns) and returns a named list mapping partition keys to their stage paths.

Usage

```
sfr_build_model_index(
  conn,
  view_fqn = "DEMO_DB.MODELS.MODEL_INDEX",
  run_id = NULL,
  stage_prefix = "@DEMO_DB.MODELS.MODELS_STAGE/"
)
```

Arguments

`conn`	An `sfr_connection` object.
`view_fqn`	Fully-qualified view name, e.g. `"DEMO_DB.MODELS.MODEL_INDEX"`.
`run_id`	Optional character. Filter to a specific training run.
`stage_prefix`	Stage path prefix prepended to RELATIVE_PATH. Defaults to `"@DEMO_DB.MODELS.MODELS_STAGE/"`

Value

A named list: `list(SKU_001 = "@stage/path.rds", ...)`.

Examples

```
## Not run:
conn <- sfr_connect()
index <- sfr_build_model_index(conn,
  view_fqn = "DEMO_DB.MODELS.MODEL_INDEX",
  run_id = "v2_S_2000sku_4c"
)
cat("Partitions:", length(index), "\n")

## End(Not run)
```

sfr_check_environment *Check the snowflakeR environment*

Description

Runs diagnostics on R, Python, and Snowflake ML dependencies.

Usage

```
sfr_check_environment()
```

Value

Invisibly returns a list of check results.

sfr_check_features *Check which snowflakeR features are available*

Description

Reports which optional features are available based on the installed version of `snowflake-ml-python`.

Usage

```
sfr_check_features(conn = NULL)
```

Arguments

conn	An `sfr_connection` object (optional). If provided, also reports the Snowflake server version.

Value

A data.frame with columns `feature`, `min_version`, `available`, and `installed`.

Examples

```
## Not run:
sfr_check_features()
#>                                 feature min_version available installed
#> 1 Core Feature Store + Model Registry       1.5.0      TRUE   1.27.0
#> 2 ML Observability (model monitoring)       1.7.1      TRUE   1.27.0
#> ...

## End(Not run)
```

sfr_collect_executor_results *Collect Executor Results from Stage*

Description

Downloads result files written by executor workers from the stage and loads them into R.

Usage

```
sfr_collect_executor_results(conn, job, stage = "DOSNOWFLAKE_STAGE")
```

Arguments

conn	An sfr_connection object.
job	A list returned by sfr_execute_r_script(), or a character string with the output path.
stage	Character. Stage name (default "DOSNOWFLAKE_STAGE").

Value

A list of result objects (one per worker).

sfr_connect *Connect to Snowflake*

Description

Creates a connection to Snowflake, returning an sfr_connection object. Supports multiple authentication methods:

Usage

```
sfr_connect(
  name = NULL,
  account = NULL,
  user = NULL,
  warehouse = NULL,
  database = NULL,
  schema = NULL,
  role = NULL,
  authenticator = NULL,
  private_key_file = NULL,
  ...,
  .use_snowflakeauth = TRUE
)
```

Arguments

name	Character. Named connection from connections.toml. If NULL, uses the [default] profile, or the only profile if there is exactly one.
account	Character. Snowflake account identifier.
user	Character. Snowflake username.
warehouse	Character. Default warehouse.
database	Character. Default database.
schema	Character. Default schema.
role	Character. Role to use.
authenticator	Character. Authentication method (e.g., "externalbrowser", "snowflake", "SNOWFLAKE_JWT", "oauth").
private_key_file	Character. Path to PEM-encoded private key for key-pair authentication. Also read from connections.toml field private_key_path.
...	Additional connection parameters passed to Snowpark session builder or snowflakeauth::snowflak
.use_snowflakeauth	Logical. Whether to use snowflakeauth for credential resolution when available. Default: TRUE.

Details

- **Auto-detect:** In Workspace Notebooks, wraps the active Snowpark session. Locally, reads connections.toml / config.toml (via snowflakeauth if installed, or directly via RcppTOML / Python toml as fallback).
- **Named connection:** Pass name to select a connection from connections.toml.
- **Explicit parameters:** Pass account, user, authenticator, etc.

Value

An sfr_connection object (S3 class).

Examples

```
## Not run:
# Default connection from connections.toml
conn <- sfr_connect()

# Named connection
conn <- sfr_connect(name = "my_profile")

# Explicit parameters with key-pair auth
conn <- sfr_connect(
  account = "xy12345.us-east-1",
  user = "MYUSER",
  private_key_file = "~/.snowflake/keys/rsa_key.p8"
)

# Explicit parameters with browser SSO
conn <- sfr_connect(
  account = "xy12345.us-east-1",
  user = "MYUSER",
```

```
  authenticator = "externalbrowser"
)

## End(Not run)
```

`sfr_create_compute_pool` *Create a compute pool*

Description

Compute pools provide GPU/CPU resources for SPCS services (model inference).

Usage

```
sfr_create_compute_pool(
  conn,
  name,
  instance_family,
  min_nodes = 1L,
  max_nodes = 1L,
  auto_resume = TRUE,
  auto_suspend_secs = 3600L,
  comment = NULL
)
```

Arguments

`conn`	An `sfr_connection` object.
`name`	Character. Compute pool name.
`instance_family`	Character. Instance type (e.g., `"CPU_X64_XS"`, `"GPU_NV_S"`).
`min_nodes`	Integer. Minimum node count. Default: `1`.
`max_nodes`	Integer. Maximum node count. Default: `1`.
`auto_resume`	Logical. Auto-resume on query. Default: `TRUE`.
`auto_suspend_secs`	Integer. Seconds of inactivity before auto-suspend. Default: `3600`.
`comment`	Character. Description.

Value

Invisibly returns `TRUE`.

sfr_create_dataset *Create a new Snowflake Dataset*

Description

Creates an empty dataset in the current database/schema. Add versions with `sfr_create_dataset_version()`.

Usage

```
sfr_create_dataset(conn, name, exist_ok = FALSE)
```

Arguments

conn	An `sfr_connection` object.
name	Character. Dataset name.
exist_ok	Logical. If `FALSE` (default), errors if the dataset already exists.

Value

An `sfr_dataset` object.

sfr_create_dataset_version *Create a dataset version from a SQL query*

Description

Materialises the result of a SQL query as an immutable dataset version.

Usage

```
sfr_create_dataset_version(
  conn,
  name,
  version,
  input_sql,
  shuffle = FALSE,
  exclude_cols = NULL,
  label_cols = NULL,
  partition_by = NULL,
  comment = NULL
)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name (must already exist).
version	Character. Version name.
input_sql	Character. SQL query whose results become the version data.
shuffle	Logical. Globally shuffle the data. Default: FALSE.
exclude_cols	Character vector. Columns to exclude during training/testing (e.g., timestamps).
label_cols	Character vector. Columns containing labels.
partition_by	Character. SQL expression for partitioning.
comment	Character. Description of this version.

Value

An sfr_dataset object with the new version info.

---

sfr_create_eai *Create an External Access Integration*

Description

EAIs are required for SPCS containers that need outbound network access (e.g., downloading R packages from CRAN during model inference).

Usage

```
sfr_create_eai(
  conn,
  name,
  allowed_network_rules,
  allowed_auth_integrations = NULL,
  enabled = TRUE,
  comment = NULL
)
```

Arguments

conn	An sfr_connection object.
name	Character. EAI name.
allowed_network_rules	Character vector. Network rule names to allow.
allowed_auth_integrations	Character vector. API authentication integration names. Default: NULL.
enabled	Logical. Whether the EAI is enabled. Default: TRUE.
comment	Character. Description.

Value

Invisibly returns TRUE.

sfr_create_entity *Create and register an entity*

Description

Creates an entity in the Snowflake Feature Store. Entities define the join keys used to look up features.

Usage

```
sfr_create_entity(fs, name, join_keys, desc = NULL)
```

Arguments

fs	An sfr_feature_store object from sfr_feature_store().
name	Character. Entity name.
join_keys	Character vector. Column names used as join keys.
desc	Character. Description.

Value

An sfr_entity object.

Examples

```
## Not run:
conn <- sfr_connect()
fs <- sfr_feature_store(conn, create = TRUE)
customer_entity <- sfr_create_entity(fs, "CUSTOMER", "CUSTOMER_ID")

## End(Not run)
```

sfr_create_feature_view *Create and register a Feature View in one step*

Description

Convenience wrapper that calls sfr_feature_view() then sfr_register_feature_view() internally.

Usage

```
sfr_create_feature_view(
  fs,
  name,
  version,
  entities,
  feature_df,
  refresh_freq = NULL,
  warehouse = NULL,
  timestamp_col = NULL,
  desc = NULL,
  features = NULL,
  feature_granularity = NULL,
  overwrite = FALSE,
  block = TRUE,
  initialize = NULL,
  refresh_mode = NULL,
  cluster_by = NULL,
  online_config = NULL
)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Name for the Feature View.
version	Character. Version name.
entities	An sfr_entity object or list of sfr_entity objects.
feature_df	A dbplyr lazy table, SQL string, or table reference defining the feature transformation logic. Named feature_df to match the Python FeatureView constructor parameter.
refresh_freq	Character. Refresh frequency (e.g., "1 hour", "30 minutes"). If NULL, creates an external (manually maintained) Feature View.
warehouse	Character. Warehouse for refreshing.
timestamp_col	Character. Timestamp column for point-in-time lookups.
desc	Character. Description.
features	List of objects from sfr_feature() defining aggregation features. Required when feature_granularity is set.
feature_granularity	Character. Time granularity for tile-based aggregation (e.g., "1h", "1 day").
overwrite	Logical. If TRUE, overwrites an existing version.
block	Logical. If TRUE (default), waits until Feature View materialisation completes before returning.
initialize	Character or NULL. Initial materialisation behaviour for a managed Feature View (e.g., "ON_CREATE" to populate on registration). When NULL, the Snowflake ML SDK default applies.
refresh_mode	Character or NULL. Refresh strategy for managed views, such as "INCREMENTAL" or "FULL". When NULL, the SDK default applies.

cluster_by	Character vector or NULL. Columns to cluster the backing dynamic table by. When NULL, no explicit CLUSTER BY is set.
online_config	An sfr_online_config() object or NULL. Online serving configuration; when NULL, online serving is not configured via this argument.

Value

A registered sfr_feature_view object.

---

sfr_create_image_repo *Create an image repository*

Description

Image repositories store container images for SPCS services.

Usage

```
sfr_create_image_repo(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Image repository name.

Value

Invisibly returns TRUE.

---

sfr_create_worker_pool *Create a persistent worker pool for queue-based dispatch*

Description

Launches a long-running SPCS service with multiple replicas that continuously poll the Hybrid Table queue for work.

Usage

```
sfr_create_worker_pool(
  conn,
  service_name,
  compute_pool,
  image_uri,
  replicas = 2L,
  job_id = "*",
  queue_fqn = "CONFIG.DOSNOWFLAKE_QUEUE",
  ...
)
```

Arguments

conn	An sfr_connection object.
service_name	Character. Name for the SPCS service.
compute_pool	Character. Name of the SPCS compute pool.
image_uri	Character. Docker image URI.
replicas	Integer. Number of worker instances (default 2).
job_id	Character. Job to process (*'' for any, default ''*).
queue_fqn	Character. Queue table name.
...	Additional options passed to the bridge.

Value

Invisibly returns the service name.

---

sfr_crew_controller_service *Create a crew Controller as an SPCS Service*

Description

For scenarios where the controller itself needs to run inside SPCS (e.g., long-running orchestration without a Workspace session), this function creates a persistent SPCS service with a TCP endpoint that workers can connect to.

Usage

```
sfr_crew_controller_service(
  conn,
  compute_pool,
  image,
  service_name = "CREW_CONTROLLER",
  port = 5555L,
  r_script_stage_path = "",
  stage = ""
)
```

Arguments

conn	An sfr_connection object.
compute_pool	Character. SPCS compute pool.
image	Character. Docker image URI.
service_name	Character. Service name (default "CREW_CONTROLLER").
port	Integer. TCP listener port (default 5555).
r_script_stage_path	Character. Optional R script for the controller to run.
stage	Character. Optional stage to mount.

Value

A list with service_name, dns_name, controller_url.

sfr_crew_launch_workers *Launch crew Workers for an Existing Controller*

Description

Launches a batch of SPCS worker containers that connect to an existing crew controller.

Usage

```
sfr_crew_launch_workers(
  conn,
  controller_url,
  compute_pool,
  image,
  n_workers = 4L,
  instance_family = "CPU_X64_S"
)
```

Arguments

conn	An sfr_connection object.
controller_url	Character. TCP URL of the controller, e.g. "tcp://crew_controller.abc123.svc.spcs.interna
compute_pool	Character. SPCS compute pool.
image	Character. Docker image URI.
n_workers	Integer. Number of workers (default 4).
instance_family	Character. Instance family (default "CPU_X64_S").

Value

A list with job_name, n_workers, status.

sfr_dbi_connection *Get an RSnowflake DBI connection from an sfr_connection*

Description

Returns an RSnowflake::SnowflakeConnection that can be used with standard DBI methods (dbGetQuery, dbWriteTable, etc.) and dbplyr. The connection is created lazily on first call and cached on the sfr_connection object.

Usage

```
sfr_dbi_connection(conn)
```

Arguments

conn	An sfr_connection object from sfr_connect().

Details

Requires the RSnowflake package to be installed.

Value

An RSnowflake::SnowflakeConnection object.

Examples

```
## Not run:
sfr_conn <- sfr_connect(name = "my_profile")
dbi_con  <- sfr_dbi_connection(sfr_conn)
DBI::dbGetQuery(dbi_con, "SELECT 1")

## End(Not run)
```

---

sfr_delete_compute_pool *Delete a compute pool*

Description

Delete a compute pool

Usage

```
sfr_delete_compute_pool(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Compute pool name.

Value

Invisibly returns TRUE.

---

sfr_delete_dataset *Delete a dataset and all its versions*

Description

Delete a dataset and all its versions

Usage

```
sfr_delete_dataset(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name.

Value

Invisibly returns TRUE.

---

sfr_delete_dataset_version *Delete a dataset version*

Description

Delete a dataset version

Usage

```
sfr_delete_dataset_version(conn, name, version)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name.
version	Character. Version to delete.

Value

Invisibly returns TRUE.

---

sfr_delete_eai *Delete an External Access Integration*

Description

Delete an External Access Integration

Usage

```
sfr_delete_eai(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. EAI name.

Value

Invisibly returns TRUE.

sfr_delete_entity *Delete an entity*

Description

Delete an entity

Usage

```
sfr_delete_entity(fs, name)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Entity name to delete.

Value

Invisibly returns TRUE.

sfr_delete_experiment *Delete an experiment*

Description

Delete an experiment

Usage

```
sfr_delete_experiment(exp, name = NULL)
```

Arguments

exp	An sfr_experiment object.
name	Character. Experiment name; defaults to exp$name.

Value

Invisibly TRUE.

sfr_delete_feature_view *Delete a Feature View*

Description

Delete a Feature View

Usage

```
sfr_delete_feature_view(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to delete.

Value

Invisibly returns TRUE.

sfr_delete_image_repo *Delete an image repository*

Description

Delete an image repository

Usage

```
sfr_delete_image_repo(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Image repository name.

Value

Invisibly returns TRUE.

sfr_delete_model *Delete a model from the registry*

Description

Delete a model from the registry

Usage

```
sfr_delete_model(reg, model_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Name of the model to delete.

Value

Invisibly returns TRUE.

sfr_delete_model_metric *Delete a metric from a model version*

Description

Delete a metric from a model version

Usage

```
sfr_delete_model_metric(reg, model_name, version_name, metric_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
metric_name	Character.

Value

Invisibly returns TRUE.

sfr_delete_model_version *Delete a specific model version*

Description

Delete a specific model version

Usage

```
sfr_delete_model_version(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

Invisibly returns TRUE.

sfr_delete_monitor *Delete a model monitor*

Description

Delete a model monitor

Usage

```
sfr_delete_monitor(reg, monitor_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
monitor_name	Character. Monitor to delete.

Value

Invisibly TRUE.

Examples

```
## Not run:
sfr_delete_monitor(reg, "MY_MON")

## End(Not run)
```

sfr_delete_run *Delete a run from the experiment*

Description

Delete a run from the experiment

Usage

```
sfr_delete_run(exp, run_name)
```

Arguments

exp	An sfr_experiment object.
run_name	Character. Name of the run to delete.

Value

Invisibly TRUE.

sfr_deploy_many_model *Deploy a Many-Model Aggregator as an SPCS Service*

Description

Creates an SPCS inference service from the registered aggregator. Uses the predict_single method (FUNCTION type) for online inference.

Usage

```
sfr_deploy_many_model(
  reg,
  model_name,
  version_name,
  service_name,
  compute_pool,
  min_instances = 1L,
  max_instances = 1L
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Model name in registry.
version_name	Character. Version label.
service_name	Character. Name for the SPCS service.
compute_pool	Character. SPCS compute pool name.
min_instances	Integer. Minimum replicas (default 1).
max_instances	Integer. Maximum replicas (default 1).

Value

Invisible. The service name.

Examples

```
## Not run:
sfr_deploy_many_model(
  reg,
  model_name = "R_SKU_FORECAST",
  version_name = "V_2k",
  service_name = "R_FORECAST_SVC",
  compute_pool = "DEMO_POOL"
)

## End(Not run)
```

---

sfr_deploy_model *Deploy a model as an SPCS service*

Description

Deploy a model as an SPCS service

Usage

```
sfr_deploy_model(
  reg,
  model_name,
  version_name,
  service_name,
  compute_pool,
  image_repo,
  max_instances = 1L,
  min_instances = NULL,
  force = FALSE,
  autocapture = FALSE,
  image_build_compute_pool = NULL,
  cpu_requests = NULL,
  memory_requests = NULL,
  gpu_requests = NULL,
  num_workers = NULL,
  max_batch_rows = NULL,
  block = TRUE,
  build_external_access_integrations = NULL
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

service_name Character. Name for the SPCS service.

compute_pool Character. Compute pool to use.

image_repo Character. Image repository.

max_instances Integer. Max service instances. Default: 1.

min_instances Integer or NULL. Minimum running service instances (supports scale-to-zero when 0). Requires snowflake-ml-python 1.25.0 or newer. Default: NULL (SDK default).

force Logical. If TRUE and the service already exists, drop it first and redeploy. Default: FALSE.

autocapture Logical. If TRUE, inference requests and responses are automatically logged to the model's inference table. Query captured data with SELECT * FROM TABLE(INFERENCE_TABLE('<m
The model must have been created after 2026-01-23 (or cloned from an older model). Default: FALSE.

image_build_compute_pool
Character or NULL. Compute pool used when the service image is built in Snowflake. When NULL, build uses the deployment pool or SDK default.

cpu_requests, memory_requests, gpu_requests
Character or NULL. Per-container resource requests as strings (CPU/memory/GPU quantities in the form accepted by the Snowflake ML service deployer, e.g. "500m" or "2" for CPU and "8Gi" for memory), forwarded to service creation.

num_workers Integer or NULL. Number of inference worker processes inside the service container. When NULL, the SDK default applies.

max_batch_rows Integer or NULL. Maximum batch size for batched inference endpoints. When NULL, the SDK default applies.

block Logical. If TRUE (default), waits until the service reaches a ready state before returning.

build_external_access_integrations
Character vector or NULL. Names of external access integrations to attach during image build for outbound network access. When NULL, none are added beyond SDK defaults.

Value

Invisibly returns a list with deployment info.

---

sfr_describe_compute_pool

Describe a compute pool

Description

Describe a compute pool

Usage

```
sfr_describe_compute_pool(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Compute pool name.

Value

A data.frame with pool properties.

---

sfr_describe_eai *Describe an External Access Integration*

Description

Describe an External Access Integration

Usage

```
sfr_describe_eai(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. EAI name.

Value

A data.frame with integration properties.

---

sfr_describe_image_repo *Describe an image repository*

Description

Describe an image repository

Usage

```
sfr_describe_image_repo(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Image repository name.

Value

A data.frame with repository properties.

sfr_describe_monitor *Describe a model monitor*

Description

Describe a model monitor

Usage

```
sfr_describe_monitor(conn, monitor_name)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.

Value

A data.frame from DESCRIBE MODEL MONITOR.

Examples

```
## Not run:
sfr_describe_monitor(conn, "MY_MON")

## End(Not run)
```

sfr_disconnect *Disconnect a Snowflake connection*

Description

Closes the underlying Snowpark session. If an RSnowflake DBI connection is also attached, it is disconnected too.

Usage

```
sfr_disconnect(conn)
```

Arguments

conn	An sfr_connection object.

Value

Invisibly returns TRUE.

`sfr_dosnowflake_build_image` *Build and push a doSnowflake worker Docker image*

Description

Generates a Dockerfile from the built-in template, optionally adding user-specified packages, then builds and pushes the image to a Snowflake image repository.

Usage

```
sfr_dosnowflake_build_image(
  conn,
  image_repo,
  tag = "latest",
  variant = c("lean", "full"),
  packages = NULL,
  docker_context = NULL
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`image_repo`	Character. Name of the Snowflake image repository (e.g. `"MY_REPO"`).
`tag`	Character. Image tag (default `"latest"`).
`variant`	Character. `"lean"` (R only, ~500 MB) or `"full"` (R + Python + reticulate, ~2 GB).
`packages`	Named list of additional packages to install: • `conda` – character vector of conda-forge package names • `cran` – character vector of CRAN package names • `pip` – character vector of pip package names
`docker_context`	Character. Path to the directory containing the Dockerfiles and worker.R. If `NULL`, uses the built-in templates from `internal/doSnowflake/docker/`.

Details

Requires Docker to be installed and running locally.

Value

The full image URI (invisibly).

Examples

```
## Not run:
conn <- sfr_connect()
uri <- sfr_dosnowflake_build_image(
  conn, "MY_REPO",
  variant = "lean",
  packages = list(cran = c("randomForest", "glmnet"))
```

```
)
# uri is now ready to pass to registerDoSnowflake(image_uri = uri)

## End(Not run)
```

`sfr_dosnowflake_setup` *Set up Snowflake infrastructure for doSnowflake*

Description

Creates the internal stage (if it doesn't exist) and validates that the compute pool and image repository are available. Call this once before using `registerDoSnowflake(mode = "tasks")` or `registerDoSnowflake(mode = "queue")` with SPCS workers.

Usage

```
sfr_dosnowflake_setup(
  conn,
  stage = "DOSNOWFLAKE_STAGE",
  compute_pool = NULL,
  image_repo = NULL
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`stage`	Character. Name of the internal stage to create (default `"DOSNOWFLAKE_STAGE"`).
`compute_pool`	Character. Name of the compute pool to validate. If `NULL`, validation is skipped.
`image_repo`	Character. Name of the image repository to validate. If `NULL`, validation is skipped.

Value

A list with `stage`, `compute_pool`, and `image_repo` names (invisibly).

Examples

```
## Not run:
conn <- sfr_connect()
sfr_dosnowflake_setup(conn,
  compute_pool = "MY_POOL",
  image_repo   = "MY_REPO"
)

## End(Not run)
```

sfr_drop_monitor_segment *Drop a segment column from a model monitor*

Description

Drop a segment column from a model monitor

Usage

```
sfr_drop_monitor_segment(conn, monitor_name, column)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.
column	Character. Segment column name (string column in source data).

Value

Invisibly TRUE.

Examples

```
## Not run:
sfr_drop_monitor_segment(conn, "MY_MON", "REGION")

## End(Not run)
```

sfr_end_run *End the active experiment run*

Description

End the active experiment run

Usage

```
sfr_end_run(exp, name = NULL)
```

Arguments

exp	An sfr_experiment object.
name	Character. Optional run name (passed to the Python SDK).

Value

Invisibly TRUE.

sfr_execute *Execute a SQL statement for side effects*

Description

Runs DDL, DML, or other SQL that doesn't return a result set. When an RSnowflake DBI connection is available, the statement is executed via the pure-R REST API path. Otherwise falls back to the Python Snowpark bridge.

Usage

```
sfr_execute(conn, sql)
```

Arguments

conn	An sfr_connection object from sfr_connect().
sql	Character. SQL statement string.

Value

Invisibly returns TRUE on success.

Examples

```
## Not run:
conn <- sfr_connect()
sfr_execute(conn, "CREATE TABLE test_table (id INT, name STRING)")

## End(Not run)
```

sfr_execute_r_script *Execute an R Script on SPCS*

Description

Launches a generic R executor on Snowflake Snowpark Container Services that loads and runs the specified R script from a mounted stage. The R script must define an entry function (default main(config)) that receives a configuration list.

Usage

```
sfr_execute_r_script(
  conn,
  r_script_stage_path,
  compute_pool,
  image_uri,
  config = list(),
  replicas = 1L,
  entry_fn = "main",
```

```
  output_mode = "stage",
  output_format = "rds",
  stage = "DOSNOWFLAKE_STAGE",
  instance_family = "CPU_X64_S",
  eai = "",
  async = TRUE,
  job_name = ""
)
```

Arguments

conn	An sfr_connection object from sfr_connect().
r_script_stage_path	Character. Path to the R script on the mounted stage, e.g. "/stage/scripts/my_script.R".
compute_pool	Character. SPCS compute pool name.
image_uri	Character. Docker image URI for the executor container.
config	Named list. Configuration passed to the R entry function as a list. Written as JSON to the stage.
replicas	Integer. Number of parallel workers (default 1).
entry_fn	Character. Name of the entry function (default "main").
output_mode	Character. "stage", "table", "stdout", or "none" (default "stage").
output_format	Character. Output file format for stage results: "rds" (default, any R object), "parquet" (needs arrow pkg), "csv", or "json"/"ndjson". Parquet, CSV, and NDJSON can be read directly by Snowflake COPY INTO or External Tables.
stage	Character. Stage to mount in the container (default "DOSNOWFLAKE_STAGE").
instance_family	Character. SPCS instance family for resource sizing (default "CPU_X64_S").
eai	Character. External Access Integration name for internet access (default "").
async	Logical. If TRUE (default), launch asynchronously. If FALSE, block until completion.
job_name	Character. Custom job service name. Auto-generated if empty.

Details

This avoids Docker image rebuilds – update the R script on the stage and re-launch.

Value

A list with job_id, job_name, replicas, status, output_path.

Examples

```
## Not run:
conn <- sfr_connect()

# Upload script
sfr_upload_r_script(conn, "optimise_sku.R")

# Launch with 4 replicas
job <- sfr_execute_r_script(
```

```
  conn,
  r_script_stage_path = "/stage/scripts/optimise_sku.R",
  compute_pool = "MY_POOL",
  image_uri = "/DB/SCHEMA/REPO/r_executor:latest",
  config = list(max_iter = 1000, tolerance = 0.01),
  replicas = 4
)
cat("Job launched:", job$job_name, "\n")

# Poll for completion
status <- sfr_executor_status(conn, job$job_name)

## End(Not run)
```

`sfr_executor_status` *Poll Executor Job Status*

Description

Checks the current status of an R executor job launched by `sfr_execute_r_script()`.

Usage

```
sfr_executor_status(conn, job_name, timeout = 3600L, poll_interval = 10L)
```

Arguments

`conn`	An `sfr_connection` object.
`job_name`	Character. The job service name returned by `sfr_execute_r_script()$job_name`.
`timeout`	Integer. Maximum seconds to wait (default 3600).
`poll_interval`	Integer. Seconds between polls (default 10).

Value

A list with `status`, `elapsed_sec`, and optionally `instances`.

`sfr_experiment` *Create or open an ML experiment*

Description

Connects experiment tracking to a Snowflake session and activates the named experiment in the underlying `snowflake-ml-python` SDK.

Usage

```
sfr_experiment(conn, name, database = NULL, schema = NULL)
```

Arguments

conn	An sfr_connection object from sfr_connect().
name	Character. Experiment name.
database	Character. Optional database context (reserved for future use).
schema	Character. Optional schema context (reserved for future use).

Value

An sfr_experiment object with fields conn, name, database, and schema.

---

sfr_experiment_from_tune

Log each tuning configuration as an experiment run

Description

Requires the **tune** package. For each distinct configuration in tune::collect_metrics() output, starts a run, logs hyperparameters as parameters and aggregated metrics via sfr_exp_log_metric().

Usage

```
sfr_experiment_from_tune(exp, tune_results, prefix = "run")
```

Arguments

exp	An sfr_experiment object.
tune_results	A tune_results object (e.g. from tune::tune_grid()).
prefix	Character prefix for generated run names.

Value

Invisibly TRUE.

---

sfr_experiment_log_best

Select the best tuning result, log a run, and register the fitted model

---

Description

Fits a **workflows** workflow with tune::select_best() on train_data, logs parameters and metrics to the experiment, then calls sfr_log_model() on the connection. Requires **tune**, **workflows**, and **parsnip**.

Usage

```
sfr_experiment_log_best(
  exp,
  tune_results,
  workflow,
  model_name,
  metric = "roc_auc",
  train_data,
  ...
)
```

Arguments

exp	An sfr_experiment object.
tune_results	A tune_results object.
workflow	An unfitted **workflows** workflow matching tune_results.
model_name	Character. Name for sfr_log_model().
metric	Character. Metric passed to tune::select_best().
train_data	Training data for the final fit.
...	Additional arguments passed to sfr_log_model().

Value

Invisibly TRUE.

---

sfr_export_model *Export model files to a local directory*

Description

Export model files to a local directory

Usage

```
sfr_export_model(
  reg,
  model_name,
  version_name,
  target_path,
  export_mode = "model"
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
target_path	Character. Local directory to export to.
export_mode	Character. Export mode (default: "model").

Value

The target path (invisibly).

`sfr_exp_download_artifact` *Download artifacts for a run*

Description

Download artifacts for a run

Usage

```
sfr_exp_download_artifact(exp, run_name, artifact_path, target_path)
```

Arguments

`exp`	An `sfr_experiment` object.
`run_name`	Character. Run name.
`artifact_path`	Character. Artifact path within the run.
`target_path`	Character. Local directory to write files into.

Value

Invisibly `TRUE`.

`sfr_exp_list_artifacts` *List artifacts for a run*

Description

List artifacts for a run

Usage

```
sfr_exp_list_artifacts(exp, run_name, artifact_path = NULL)
```

Arguments

`exp`	An `sfr_experiment` object.
`run_name`	Character. Run name.
`artifact_path`	Character. Optional prefix filter.

Value

A value from the Python SDK (typically converted by `reticulate`).

sfr_exp_log_artifact *Log a local file as a run artifact*

Description

Log a local file as a run artifact

Usage

```
sfr_exp_log_artifact(exp, local_path, artifact_path = NULL)
```

Arguments

exp	An sfr_experiment object.
local_path	Character. Path to the file on disk.
artifact_path	Character. Optional path prefix inside the artifact store.

Value

Invisibly TRUE.

sfr_exp_log_metric *Log a single metric on the active run*

Description

Log a single metric on the active run

Usage

```
sfr_exp_log_metric(exp, key, value, step = 0L)
```

Arguments

exp	An sfr_experiment object.
key	Character. Metric name.
value	Numeric metric value.
step	Integer step (e.g. training step or epoch).

Value

Invisibly TRUE.

sfr_exp_log_metrics *Log multiple metrics on the active run*

Description

Log multiple metrics on the active run

Usage

```
sfr_exp_log_metrics(exp, ..., step = 0L)
```

Arguments

exp	An sfr_experiment object.
...	Named metric values.
step	Integer step passed to the Python SDK.

Value

Invisibly TRUE.

sfr_exp_log_model *Log a model to the active run*

Description

The model object is passed through reticulate to the Snowflake ML experiment API. Optional signatures and sample_input_data can be supplied as named arguments in ....

Usage

```
sfr_exp_log_model(exp, model, model_name, ...)
```

Arguments

exp	An sfr_experiment object.
model	A model object acceptable to the Python SDK.
model_name	Character. Registered name for the model within the run.
...	Optional signatures and sample_input_data (see Snowflake ML ExperimentTracking.log_mod documentation).

Value

Invisibly TRUE.

sfr_exp_log_param *Log a single parameter on the active run*

Description

Log a single parameter on the active run

Usage

```
sfr_exp_log_param(exp, key, value)
```

Arguments

exp	An sfr_experiment object.
key	Character. Parameter name.
value	Parameter value (scalar; converted for Python).

Value

Invisibly TRUE.

sfr_exp_log_params *Log multiple parameters on the active run*

Description

All arguments in ... must be named; they are passed as a single parameter dictionary to the Python SDK.

Usage

```
sfr_exp_log_params(exp, ...)
```

Arguments

exp	An sfr_experiment object.
...	Named parameter values.

Value

Invisibly TRUE.

sfr_feature *Create an aggregation Feature definition*

Description

Defines a single aggregation feature for tile-based Feature Views. Pass a list of `sfr_feature` objects to `sfr_feature_view()` or `sfr_create_feature_view()` via the `features` parameter when using `feature_granularity`.

Usage

```
sfr_feature(name, dtype, agg, window)
```

Arguments

`name`	Character. Column name in the source data to aggregate.
`dtype`	Character. Snowflake data type (e.g., `"FLOAT"`, `"NUMBER"`).
`agg`	Character. Aggregation function (`"SUM"`, `"AVG"`, `"MIN"`, `"MAX"`, `"COUNT"`, `"STDDEV"`, `"VARIANCE"`).
`window`	Character. Time window for the aggregation (e.g., `"1 hour"`, `"1 day"`, `"7 days"`).

Value

An `sfr_feature` object.

Examples

```
## Not run:
feat <- sfr_feature("AMOUNT", "FLOAT", "SUM", "1 hour")

## End(Not run)
```

sfr_feature_store *Connect to the Feature Store*

Description

Creates or connects to a Snowflake Feature Store in the specified database/schema. Returns an `sfr_feature_store` object used by other Feature Store functions.

Usage

```
sfr_feature_store(
  conn,
  database = NULL,
  schema = NULL,
  warehouse = NULL,
  create = FALSE,
  default_iceberg_external_volume = NULL
)
```

Arguments

conn	An sfr_connection object from sfr_connect().
database	Character. Database for the Feature Store. Defaults to the connection's current database.
schema	Character. Schema for the Feature Store. Defaults to the connection's current schema.
warehouse	Character. Default warehouse for Feature Store compute. Defaults to the connection's current warehouse.
create	Logical. If TRUE, creates the Feature Store schema and tags if they don't exist. Default: FALSE.
default_iceberg_external_volume	Character or NULL. Default external volume name for Iceberg-backed Feature Store tables. Stored on the returned object; when NULL, account or SDK defaults apply.

Value

An sfr_feature_store object.

Examples

```
## Not run:
conn <- sfr_connect()
fs <- sfr_feature_store(conn, create = TRUE)

## End(Not run)
```

---

sfr_feature_view *Create a local draft Feature View*

Description

Creates an sfr_feature_view object locally without registering it in Snowflake. Use sfr_register_feature_view() to materialise it, or use sfr_create_feature_view() for a one-step convenience.

Usage

```
sfr_feature_view(
  name,
  entities,
  feature_df,
  refresh_freq = NULL,
  warehouse = NULL,
  timestamp_col = NULL,
  desc = NULL,
  features = NULL,
  feature_granularity = NULL,
  initialize = NULL,
  refresh_mode = NULL,
```

```
  cluster_by = NULL,
  online_config = NULL
)
```

Arguments

name	Character. Name for the Feature View.
entities	An sfr_entity object or list of sfr_entity objects.
feature_df	A dbplyr lazy table, SQL string, or table reference defining the feature transformation logic. Named feature_df to match the Python FeatureView constructor parameter.
refresh_freq	Character. Refresh frequency (e.g., "1 hour", "30 minutes"). If NULL, creates an external (manually maintained) Feature View.
warehouse	Character. Warehouse for refreshing.
timestamp_col	Character. Timestamp column for point-in-time lookups.
desc	Character. Description.
features	List of objects from sfr_feature() defining aggregation features. Required when feature_granularity is set.
feature_granularity	Character. Time granularity for tile-based aggregation (e.g., "1h", "1 day").
initialize	Character or NULL. Initial materialisation behaviour for a managed Feature View (e.g., "ON_CREATE" to populate on registration). When NULL, the Snowflake ML SDK default applies.
refresh_mode	Character or NULL. Refresh strategy for managed views, such as "INCREMENTAL" or "FULL". When NULL, the SDK default applies.
cluster_by	Character vector or NULL. Columns to cluster the backing dynamic table by. When NULL, no explicit CLUSTER BY is set.
online_config	An sfr_online_config() object or NULL. Online serving configuration; when NULL, online serving is not configured via this argument.

Value

An sfr_feature_view object (local draft, not yet registered).

See Also

sfr_register_feature_view(), sfr_create_feature_view()

---

sfr_fqn *Build a fully qualified table name*

Description

Constructs DATABASE.SCHEMA.TABLE_NAME from the connection context. Ensures all notebook table references are fully qualified as recommended by Snowflake for Workspace Notebooks.

Usage

```
sfr_fqn(conn, table_name, database = NULL, schema = NULL)
```

Arguments

conn	An sfr_connection object.
table_name	Character. Unqualified table name.
database	Character. Database name override. Default: connection's current database.
schema	Character. Schema name override. Default: connection's current schema.

Details

By default, uses the connection's current database and schema. Pass database and/or schema to override – useful when referencing tables in a different schema or database without changing the session context.

Value

Character. Fully qualified table name.

Examples

```
## Not run:
# Uses connection's current database/schema
sfr_fqn(conn, "MY_TABLE")
# => "MY_DB.MY_SCHEMA.MY_TABLE"

# Override schema (same database)
sfr_fqn(conn, "MY_TABLE", schema = "OTHER_SCHEMA")
# => "MY_DB.OTHER_SCHEMA.MY_TABLE"

# Override both
sfr_fqn(conn, "MY_TABLE", database = "OTHER_DB", schema = "RAW")
# => "OTHER_DB.RAW.MY_TABLE"

## End(Not run)
```

---

sfr_fv_fqn *Get the fully qualified name of a Feature View*

Description

Get the fully qualified name of a Feature View

Usage

```
sfr_fv_fqn(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.

Value

A character string with the fully qualified name.

---

sfr_fv_lineage *Get lineage for a Feature View*

Description

Get lineage for a Feature View

Usage

```
sfr_fv_lineage(fs, name, version, direction = "both", domain_filter = NULL)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.
direction	Character. "upstream", "downstream", or "both".
domain_filter	Character vector. Optional domain filter.

Value

A data.frame or character representation of lineage.

---

sfr_fv_query *Get the underlying SQL query for a Feature View*

Description

Get the underlying SQL query for a Feature View

Usage

```
sfr_fv_query(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.

Value

A character string with the SQL query.

sfr_fv_to_df *Convert a Feature View to a data.frame*

Description

Convert a Feature View to a data.frame

Usage

```
sfr_fv_to_df(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.

Value

A data.frame with the Feature View data.

sfr_generate_dataset *Generate an immutable, versioned Dataset from Feature Views*

Description

Creates a Snowflake ML Dataset that participates in ML Lineage (Feature View -> Dataset -> Model). The returned data.frame carries dataset_name and dataset_version attributes that sfr_log_model() uses to wire the lineage chain.

Usage

```
sfr_generate_dataset(
  fs,
  name,
  spine,
  features,
  version = NULL,
  spine_timestamp_col = NULL,
  spine_label_cols = NULL,
  desc = "",
  exclude_columns = NULL,
  include_feature_view_timestamp_col = FALSE,
  auto_prefix = FALSE,
  output_type = NULL,
  join_method = NULL
)
```

Arguments

fs	An sfr_feature_store object.
name	Dataset name.
spine	A data.frame, SQL string, or dbplyr lazy table providing entity keys and (optionally) labels.
features	A list of registered sfr_feature_view objects or lists with name and version.
version	Dataset version string. When NULL (the default), Snowflake auto-generates a version.
spine_timestamp_col	Optional timestamp column for point-in-time correctness.
spine_label_cols	Optional character vector of label column names.
desc	Optional description for the dataset.
exclude_columns	Character vector or NULL. Columns to omit from the generated dataset. Default: NULL.
include_feature_view_timestamp_col	Logical. If TRUE, includes Feature View timestamp columns. Default: FALSE.
auto_prefix	Logical. If TRUE, prefixes feature columns with the Feature View name. Default: FALSE.
output_type	Character or NULL. Dataset storage or output format hint forwarded to FeatureStore.generate_data When NULL, the SDK default applies.
join_method	Character or NULL. Join strategy for spine and features (e.g., "AS_OF"). When NULL, the SDK default applies.

Value

A data.frame of training data with attributes dataset_name and dataset_version attached. Pass this object to sfr_log_model() via training_dataset to complete Feature View -> Dataset -> Model lineage.

See Also

sfr_generate_training_data(), sfr_log_model()

Examples

```
## Not run:
conn <- sfr_connect()
fs <- sfr_feature_store(conn)

ds <- sfr_generate_dataset(
  fs,
  name = "CHURN_TRAINING",
  spine = "SELECT customer_id, label FROM labels_table",
  features = list(
    list(name = "CUSTOMER_FEATURES", version = "v1")
  ),
  version = "v1",
  spine_label_cols = "label"
)
```

```
# Train a model using ds as a plain data.frame ...
# Then log with lineage:
sfr_log_model(conn, model, "CHURN_MODEL",
              training_dataset = ds)

## End(Not run)
```

`sfr_generate_training_data` *Generate training data from Feature Views*

Description

Joins spine (label) data with registered Feature Views to produce a training dataset. Supports point-in-time correct joins when `spine_timestamp_col` is provided.

Usage

```
sfr_generate_training_data(
  fs,
  spine,
  features,
  spine_timestamp_col = NULL,
  spine_label_cols = NULL,
  save_as = NULL,
  exclude_columns = NULL,
  include_feature_view_timestamp_col = FALSE,
  auto_prefix = FALSE,
  join_method = NULL
)
```

Arguments

`fs`	An `sfr_feature_store` object.
`spine`	A data.frame, dbplyr lazy table, or SQL string for the spine (label) data.
`features`	A list of `sfr_feature_view` objects (must be registered, with `name` and `version` attributes), or a list of lists with `name` and `version` elements.
`spine_timestamp_col`	Character. Timestamp column in the spine for point-in-time joins.
`spine_label_cols`	Character vector. Label columns in the spine.
`save_as`	Character. Optional table name to materialise results.
`exclude_columns`	Character vector or `NULL`. Feature or spine column names to drop from the joined result. Default: `NULL` (keep all).
`include_feature_view_timestamp_col`	Logical. If `TRUE`, includes each Feature View's timestamp column in the output. Default: `FALSE`.

auto_prefix	Logical. If TRUE, prefixes feature columns with the Feature View name to avoid name clashes. Default: FALSE.
join_method	Character or NULL. Join strategy passed to the SDK (e.g., "AS_OF"). When NULL, the SDK chooses a default.

Value

A data.frame with training data.

Examples

```
## Not run:
conn <- sfr_connect()
fs <- sfr_feature_store(conn)

training_data <- sfr_generate_training_data(
  fs,
  spine = "SELECT customer_id, label FROM labels_table",
  features = list(
    list(name = "CUSTOMER_FEATURES", version = "v1"),
    list(name = "TRANSACTION_FEATURES", version = "v1")
  ),
  spine_label_cols = "label"
)

## End(Not run)
```

---

sfr_get_dataset *Load an existing Snowflake Dataset*

Description

Load an existing Snowflake Dataset

Usage

```
sfr_get_dataset(conn, name)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name.

Value

An sfr_dataset object.

sfr_get_entity *Get a registered entity*

Description

Get a registered entity

Usage

```
sfr_get_entity(fs, name)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Entity name.

Value

An sfr_entity object.

sfr_get_feature_view *Get a Feature View by name and version*

Description

Get a Feature View by name and version

Usage

```
sfr_get_feature_view(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to retrieve.

Value

An sfr_feature_view object.

sfr_get_model *Get a model reference from the registry*

Description

Get a model reference from the registry

Usage

```
sfr_get_model(reg, model_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Name of the model.

Value

An sfr_model object.

sfr_get_model_metric *Get a single metric from a model version*

Description

Get a single metric from a model version

Usage

```
sfr_get_model_metric(reg, model_name, version_name, metric_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
metric_name	Character.

Value

The metric value.

sfr_get_model_task *Get the task type of a model version*

Description

Get the task type of a model version

Usage

```
sfr_get_model_task(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

Character string of the task type, or NULL.

sfr_get_model_version *Get a specific model version*

Description

Get a specific model version

Usage

```
sfr_get_model_version(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Name of the model.
version_name	Character. Version to retrieve.

Value

An sfr_model_version object.

sfr_get_monitor *Get a model monitor reference*

Description

Get a model monitor reference

Usage

```
sfr_get_monitor(reg, name = NULL, model_name = NULL, version_name = NULL)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
name	Character. Monitor name (if provided, model_name and version_name are ignored).
model_name	Character. Model name (use with version_name).
version_name	Character. Model version name.

Value

A list with at least name and status.

Examples

```
## Not run:
sfr_get_monitor(reg, name = "MY_MON")
sfr_get_monitor(reg, model_name = "M", version_name = "v1")

## End(Not run)
```

sfr_get_refresh_history *Get refresh history for a Feature View*

Description

Returns information about past refreshes of a managed Feature View, including timestamps, status, and refresh type (incremental vs full).

Usage

```
sfr_get_refresh_history(
  fs,
  name,
  version,
  verbose = FALSE,
  store_type = "OFFLINE"
)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to inspect.
verbose	Logical. If TRUE, returns more detailed history. Default: FALSE.
store_type	Character. Which store to report history for: "OFFLINE" (default) or "ONLINE".

Value

A data.frame with refresh history.

---

sfr_get_service_status

Get the current status of an SPCS service

Description

Queries SYSTEM$GET_SERVICE_STATUS() and returns a structured result. Useful for one-off status checks without starting a polling loop.

Usage

```
sfr_get_service_status(reg, service_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
service_name	Character. Name of the SPCS service.

Value

A list with:

status Character. Overall service status (e.g. "READY", "PENDING", "FAILED").

message Character. Human-readable status message, or NA.

containers A data.frame of per-container statuses, or NULL if the response couldn't be parsed as a table.

fqn Character. The fully-qualified service name used in the query.

Examples

```
## Not run:
st <- sfr_get_service_status(reg, "my_svc")
st$status   # "READY"
st$message  # "Running"

## End(Not run)
```

sfr_has_connection *Check if a Snowflake connection can be established*

Description

Useful for `@examplesIf` and test guards.

Usage

```
sfr_has_connection(...)
```

Arguments

`...`	Arguments passed to `sfr_connect()`.

Value

Logical.

sfr_input_cols *Infer input column schema from a data.frame*

Description

Builds the `input_cols` list expected by `sfr_log_model()` by inspecting column types in a data.frame. Columns listed in `exclude` (typically the target variable and ID columns) are omitted.

Usage

```
sfr_input_cols(data, exclude = character(0))
```

Arguments

`data`	A data.frame (e.g. the training data used to fit the model).
`exclude`	Character vector of column names to exclude (case-insensitive).

Value

A named list mapping column names to type strings (`"double"`, `"integer"`, `"string"`, `"boolean"`).

Examples

```
## Not run:
cols <- sfr_input_cols(training_data,
                       exclude = c("defaulted", "customer_id", "application_id"))
mv <- sfr_log_model(reg, model, "MY_MODEL", input_cols = cols, ...)

## End(Not run)
```

`sfr_install_python_deps` *Install Python dependencies for snowflakeR*

Description

Creates a conda environment with all required Python packages for `snowflakeR` to function. This environment is shared between `reticulate` and `rpy2`, avoiding duplicate installations.

Usage

```
sfr_install_python_deps(
  method = "conda",
  envname = "r-snowflakeR",
  python_version = "3.11"
)
```

Arguments

`method`	Character. Installation method: `"conda"` (recommended), `"pip"`, or `"auto"`. Default: `"conda"`.
`envname`	Character. Name of the conda/virtual environment. Default: `"r-snowflakeR"`.
`python_version`	Character. Python version to use. Default: `"3.11"`.

Details

The recommended setup is a single conda environment that both `reticulate` and `rpy2` share. This avoids installing packages twice. In Workspace Notebooks, the micromamba environment already serves this purpose.

Value

Invisibly returns `TRUE` on success.

Examples

```
## Not run:
sfr_install_python_deps()

## End(Not run)
```

sfr_list_compute_pools *List compute pools*

Description

List compute pools

Usage

```
sfr_list_compute_pools(conn)
```

Arguments

conn	An sfr_connection object.

Value

A data.frame.

sfr_list_datasets *List datasets in the current database/schema*

Description

List datasets in the current database/schema

Usage

```
sfr_list_datasets(conn)
```

Arguments

conn	An sfr_connection object.

Value

A data.frame with dataset information.

sfr_list_dataset_versions *List versions of a dataset*

Description

List versions of a dataset

Usage

```
sfr_list_dataset_versions(conn, name, detailed = FALSE)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name.
detailed	Logical. If TRUE, returns a data.frame with full metadata. Default: FALSE (returns character vector of version names).

Value

A character vector (default) or data.frame (if detailed = TRUE).

sfr_list_eais *List External Access Integrations*

Description

List External Access Integrations

Usage

```
sfr_list_eais(conn)
```

Arguments

conn	An sfr_connection object.

Value

A data.frame.

sfr_list_entities *List entities in the Feature Store*

Description

List entities in the Feature Store

Usage

```
sfr_list_entities(fs)
```

Arguments

fs	An sfr_feature_store object.

Value

A data.frame listing entities.

sfr_list_feature_views *List Feature Views*

Description

List Feature Views

Usage

```
sfr_list_feature_views(fs, entity_name = NULL, feature_view_name = NULL)
```

Arguments

fs	An sfr_feature_store object.
entity_name	Character. Filter by entity name.
feature_view_name	Character. Filter by Feature View name.

Value

A data.frame listing Feature Views.

sfr_list_fields *List columns/fields of a table*

Description

List columns/fields of a table

Usage

```
sfr_list_fields(conn, table_name)
```

Arguments

conn	An sfr_connection object.
table_name	Character. Name of the table.

Value

A data.frame with column names and types.

sfr_list_fv_columns *List columns for a Feature View*

Description

List columns for a Feature View

Usage

```
sfr_list_fv_columns(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.

Value

A data.frame with column metadata.

sfr_list_image_repos *List image repositories*

Description

List image repositories

Usage

```
sfr_list_image_repos(conn)
```

Arguments

conn	An sfr_connection object.

Value

A data.frame.

sfr_list_services *List services deployed for a model version*

Description

List services deployed for a model version

Usage

```
sfr_list_services(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

A data.frame listing services.

sfr_list_tables *List tables in the current database/schema*

Description

List tables in the current database/schema

Usage

```
sfr_list_tables(conn, database = NULL, schema = NULL)
```

Arguments

conn	An sfr_connection object.
database	Character. Database to query. Default: connection's current.
schema	Character. Schema to query. Default: connection's current.

Value

A character vector of table names.

sfr_load_fvs_from_dataset *Load Feature Views associated with a Dataset*

Description

Load Feature Views associated with a Dataset

Usage

```
sfr_load_fvs_from_dataset(fs, dataset_name, dataset_version)
```

Arguments

fs	An sfr_feature_store object.
dataset_name	Character. Dataset name.
dataset_version	Character. Dataset version.

Value

A list of lists with name and version elements.

sfr_load_notebook_config *Load notebook configuration and set execution context*

Description

Reads a notebook_config.yaml file and applies the execution context (warehouse, database, schema) to the connection. This ensures consistent context across all notebook cells.

Usage

```
sfr_load_notebook_config(conn, config_path = "notebook_config.yaml")
```

Arguments

conn	An sfr_connection object.
config_path	Character. Path to the YAML config file. Default: "notebook_config.yaml" in the current directory.

Details

Copy notebook_config.yaml.template to notebook_config.yaml and edit with your values before running.

Value

The updated sfr_connection object (invisibly). You **must** reassign: conn <- sfr_load_notebook_config(conn).

sfr_log_executor *Register a Generic R Executor in Model Registry*

Description

Registers an R script as a TABLE_FUNCTION model version in Snowflake Model Registry. The script must define an entry function that accepts a data.frame and returns a data.frame. The function is called once per partition.

Usage

```
sfr_log_executor(
  reg,
  model_name,
  version_name = NULL,
  r_script_path,
  partition_columns = "SKU",
  r_packages = character(0),
  entry_fn = "main",
  diagnostic_mode = FALSE,
  input_columns = NULL,
```

```
  output_columns = NULL,
  conda_deps = NULL,
  pip_deps = NULL,
  comment = NULL
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Model name in the registry.
version_name	Character. Version label. If NULL, auto-generated from timestamp.
r_script_path	Character. Path to the R script on local disk. Must define the entry function.
partition_columns	Character vector. Column names for partitioning (default "SKU").
r_packages	Character vector. R packages to load before sourcing the script (default empty).
entry_fn	Character. Name of the R function to call per partition (default "main").
diagnostic_mode	Logical. If TRUE, the executor returns only diagnostic columns (status, timing, error) instead of the R function's result. Useful for long-running tasks that write results elsewhere.
input_columns	Named list of column definitions. Each element is list(name = "COL", type = "STRING"). If NULL, partition columns are used as STRING inputs.
output_columns	Named list of output column definitions. If NULL, diagnostic schema is used.
conda_deps	Character vector. Additional conda dependencies.
pip_deps	Character vector. Additional pip dependencies.
comment	Character. Optional comment.

Details

The registered model can be invoked from SQL:

```
SELECT * FROM TABLE(
  MY_EXECUTOR!execute(
    SELECT SKU, PARAMS FROM WORK_TABLE
  ) OVER (PARTITION BY SKU)
)
```

Value

A list with model_name, version_name, registration_time_s.

Examples

```
## Not run:
conn <- sfr_connect()

# Register a simple executor
result <- sfr_log_executor(
  conn,
  model_name = "SKU_OPTIMISER",
```

```
  r_script_path = "optimise_sku.R",
  partition_columns = "SKU",
  r_packages = c("dplyr", "nloptr"),
  entry_fn = "optimise",
  diagnostic_mode = TRUE
)

# Now callable from SQL:
# SELECT * FROM TABLE(SKU_OPTIMISER!execute(
#   SELECT SKU, PARAMS FROM WORK_MANIFEST
# ) OVER (PARTITION BY SKU))

## End(Not run)
```

`sfr_log_many_model` *Register a Many-Model Forecast Aggregator*

Description

Registers a single `CustomModel` in Snowflake Model Registry that dispatches inference to per-partition `.rds` files stored on an internal stage. The model supports both:

- `predict` (TABLE_FUNCTION / `@partitioned_api`) for `run_batch()` batch inference.
- `predict_single` (FUNCTION / `@inference_api`) for SPCS service deployment.

Usage

```
sfr_log_many_model(
  reg,
  model_name,
  version_name,
  model_index,
  partition_columns = "SKU",
  r_packages = "forecast",
  horizon = 12L,
  comment = NULL
)
```

Arguments

`reg`	An `sfr_model_registry` or `sfr_connection` object.
`model_name`	Character. Model name in the registry.
`version_name`	Character. Version label.
`model_index`	Named list from `sfr_build_model_index()`: `list(partition_key = "stage_path", ...)`.
`partition_columns`	Character vector. Column names forming the partition key. Default `"SKU"`.
`r_packages`	Character vector. R packages loaded at predict time. Default `"forecast"`.
`horizon`	Integer. Forecast horizon. Default 12.
`comment`	Character. Optional comment attached to the version.

Details

Registration is O(1) regardless of partition count: ~13 seconds for 2,000 or 10,000 partitions.

Value

A list with `model_name`, `version_name`, `n_partitions`, `registration_time_s`.

Examples

```
## Not run:
conn <- sfr_connect()
reg  <- sfr_registry(conn, database = "DEMO_DB", schema = "MODELS")
index <- sfr_build_model_index(conn, run_id = "v2_S_2000sku_4c")

result <- sfr_log_many_model(
  reg,
  model_name = "R_SKU_FORECAST",
  version_name = "V_2k",
  model_index = index,
  partition_columns = "SKU",
  horizon = 12
)
cat("Registered in", result$registration_time_s, "seconds\n")

## End(Not run)
```

---

sfr_log_model *Log an R model to the Snowflake Model Registry*

Description

Saves the R model to an `.rds` file, auto-generates a Python `CustomModel` wrapper (using rpy2), and registers it in the Snowflake Model Registry.

Usage

```
sfr_log_model(
  reg,
  model,
  model_name,
  version_name = NULL,
  predict_fn = "predict",
  predict_pkgs = character(0),
  predict_body = NULL,
  input_cols = NULL,
  output_cols = NULL,
  conda_deps = NULL,
  conda_channel = NULL,
  pip_requirements = NULL,
  target_platforms = "SNOWPARK_CONTAINER_SERVICES",
  comment = NULL,
  metrics = NULL,
```

```
  sample_input = NULL,
  training_dataset = NULL,
  pin_versions = TRUE,
  task = NULL,
  user_files = NULL,
  code_paths = NULL,
  resource_constraint = NULL,
  python_version = NULL,
  ...
)
```

Arguments

reg	An sfr_model_registry object from sfr_model_registry(), or an sfr_connection object from sfr_connect() (uses session defaults).
model	An R model object (anything that can be saveRDS()'d).
model_name	Character. Name for the model in the registry.
version_name	Character. Optional version name (auto-generated if NULL).
predict_fn	Character. R function name for inference. Default: "predict".
predict_pkgs	Character vector. R packages needed at inference time. **These packages must be available on conda-forge** (as r-<pkgname>) because the SPCS inference container installs packages exclusively from conda-forge. Packages installed from CRAN or GitHub in Workspace will NOT be available in the container. A warning is emitted if any packages appear to be missing from conda-forge.
predict_body	Character. Optional custom R code for prediction (advanced). Use template variables {{MODEL}}, {{INPUT}}, {{UID}}, {{N}}. Instead of writing raw template strings, use sfr_predict_body() to convert a normal R function to this format.
input_cols	Named list mapping input column names to types. Valid types: "integer", "double", "string", "boolean".
output_cols	Named list mapping output column names to types.
conda_deps	Character vector. Conda packages for the model environment. Defaults include r-base, rpy2>=3.5, and numpy<2.0. When pin_versions = TRUE (the default), exact version pins are auto-generated from the current R session for r-base and all packages in predict_pkgs (plus critical sub-dependencies for tidymodels). Any explicit version constraints you provide in conda_deps take precedence over auto-pins. **Package version warning:** Without numpy<2.0, adding r-base and rpy2 causes the conda solver to pick Python 3.12 + numpy 2.x, which crashes the inference server (recarray has no attribute fillna). If you override conda_deps, ensure you include numpy<2.0.
conda_channel	Character. Optional conda channel to force for all dependencies (e.g. "conda-forge"). When set, every entry in conda_deps that does not already contain a :: channel prefix is automatically prefixed with <conda_channel>::. This ensures the SPCS inference container resolves packages exclusively from the specified channel instead of the default Snowflake Anaconda Channel. **Legal / compliance note:** The default Snowflake Anaconda Channel is licensed by Snowflake for use within Snowflake. Set conda_channel = "conda-forge" if your organisation requires that only community-licensed (conda-forge) packages are used, avoiding any dependency on Anaconda Inc. commercial channels.

`pip_requirements`
: Character vector. Additional pip packages.

`target_platforms`
: Character. One of `"SNOWPARK_CONTAINER_SERVICES"`, `"WAREHOUSE"`, or both. Default: `"SNOWPARK_CONTAINER_SERVICES"`.

Important: R models require `rpy2` and `r-base` at inference time. These packages are **not available** in the Snowflake warehouse Anaconda channel, so `"WAREHOUSE"` inference is not currently supported for R models. Use `"SNOWPARK_CONTAINER_SERVICES"` (which installs packages in a container) or test locally with `sfr_predict_local()`.

`comment`
: Character. Description of the model.

`metrics`
: Named list. Metrics to attach to the model version.

`sample_input`
: A data.frame. Optional sample input for signature validation.

`training_dataset`
: A data.frame returned by `sfr_generate_dataset()`. When provided, the Dataset-backed Snowpark DataFrame is used as `sample_input_data` in the Python registry, completing the Feature View -> Dataset -> Model lineage chain visible in Snowsight. Overrides `sample_input` when both are provided.

`pin_versions`
: Logical. If `TRUE` (the default), automatically pins the R version and all `predict_pkgs` (plus critical tidymodels sub-dependencies) to their currently installed versions. This ensures the SPCS container environment exactly matches the training environment, preventing deserialization failures such as `hardhat::forge()` blueprint mismatches. Set to `FALSE` to use floating version constraints.

`task`
: Character or NULL. Model task type (e.g. `"TABULAR_REGRESSION"`, `"TABULAR_BINARY_CLASSIFICA`

`user_files`
: Character vector or NULL. Paths to additional files to include in the model artifact.

`code_paths`
: Character vector or NULL. Paths to R scripts to include for custom inference logic.

`resource_constraint`
: List or NULL. SPCS resource constraints for inference containers (e.g. `list(cpu = "2", memory = "4Gi")`).

`python_version`
: Character or NULL. Python version for the model environment. Defaults to the version used during training.

`...`
: Additional arguments passed to the underlying Python `Registry.log_model()`.

Value

An `sfr_model_version` object.

See Also

`sfr_predict_body()` for converting R functions to `predict_body` templates, `sfr_generate_dataset()` for lineage-aware training data, `sfr_predict_local()`, `sfr_predict()`, `sfr_show_models()`

Examples

```
## Not run:
# Simple model (default predict)
conn <- sfr_connect()
model <- lm(mpg ~ wt, data = mtcars)
mv <- sfr_log_model(conn, model, model_name = "MTCARS_MPG",
```

```
                    input_cols = list(wt = "double"),
                    output_cols = list(prediction = "double"))

# Custom predict logic via sfr_predict_body()
my_predict <- function(model, input) {
  nd     <- as.matrix(input[, c("X1", "X2"), drop = FALSE])
  pred   <- predict(model, newdata = nd)
  result <- data.frame(prediction = as.numeric(pred))
}
mv <- sfr_log_model(conn, model, model_name = "MY_MODEL",
                    predict_body = sfr_predict_body(my_predict),
                    input_cols = list(X1 = "double", X2 = "double"),
                    output_cols = list(prediction = "double"))

## End(Not run)
```

---

sfr_ml_version *Get installed snowflake-ml-python version*

Description

Get installed snowflake-ml-python version

Usage

```
sfr_ml_version()
```

Value

A numeric_version object, or NULL if the package is not installed or Python is not available.

Examples

```
## Not run:
sfr_ml_version()
#> [1] '1.27.0'

## End(Not run)
```

---

sfr_models *List Model objects in the registry*

Description

List Model objects in the registry

Usage

```
sfr_models(reg)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.

Value

A data.frame with model metadata.

---

sfr_model_description *Get or set model version description*

Description

Get or set model version description

Usage

```
sfr_model_description(reg, model_name, version_name, desc = NULL)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
desc	Character. If provided, sets the description. If NULL, returns current.

Value

The description string.

---

sfr_model_info *Query model versions from information schema (SQL-direct)*

Description

Returns metadata about model versions from the Snowflake information schema. Useful for auditing, comparing versions, and checking model attributes without loading the Python SDK.

Usage

```
sfr_model_info(conn, model_name = NULL, database = NULL)
```

Arguments

conn	An sfr_connection object from sfr_connect().
model_name	Character. Filter to a specific model. If NULL, returns all models in the database.
database	Character. Database to query. Defaults to the connection's current database.

Value

A data.frame with model version metadata.

sfr_model_lineage *Get lineage for a model version*

Description

Get lineage for a model version

Usage

```
sfr_model_lineage(
  reg,
  model_name,
  version_name,
  direction = "both",
  domain_filter = NULL
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
direction	Character. "upstream", "downstream", or "both".
domain_filter	Character vector.

Value

Lineage information.

sfr_model_registry *Connect to the Model Registry*

Description

Creates an sfr_model_registry object that targets a specific database/schema for model operations. If you want the registry to use the session's current database/schema, pass an sfr_connection directly to registry functions instead.

Usage

```
sfr_model_registry(
  conn,
  database = NULL,
  schema = NULL,
  options = NULL,
  conda_channel = NULL,
  conda_channel_strict = FALSE
)
```

Arguments

conn	An sfr_connection object from sfr_connect().
database	Character. Database for the Model Registry. Defaults to the connection's current database.
schema	Character. Schema for the Model Registry. Defaults to the connection's current schema.
options	Named list. Options forwarded to Registry(session, options=...). For example, list(enable_monitoring = TRUE) enables ML Observability.
conda_channel	Character. Default conda channel for all models logged through this registry (e.g. "conda-forge"). When set, every sfr_log_model() call inherits this channel unless explicitly overridden. Defaults to the SFR_CONDA_CHANNEL environment variable if set, otherwise NULL (Snowflake Anaconda Channel).
conda_channel_strict	Logical. If TRUE, the conda_channel set on this registry cannot be overridden or bypassed by individual sfr_log_model() calls. Any attempt to pass a different conda_channel or to include deps with a mismatched channel:: prefix will raise an error. Defaults to the SFR_CONDA_CHANNEL_STRICT environment variable ("true" / "1") if set, otherwise FALSE. **Use case:** Platform / compliance teams set conda_channel_strict = TRUE to guarantee that all models registered through this registry resolve packages exclusively from the approved channel (e.g. conda-forge), preventing accidental use of Anaconda Inc. commercial channels.

Value

An sfr_model_registry object.

Examples

```
## Not run:
conn <- sfr_connect()

# Option A: Explicit registry with target db/schema
reg <- sfr_model_registry(conn, database = "ML_DB", schema = "MODELS")
sfr_log_model(reg, model, "MY_MODEL", ...)

# Option B: Use connection directly (session's current db/schema)
sfr_log_model(conn, model, "MY_MODEL", ...)

# Option C: Enforce conda-forge for all models (compliance mode)
reg <- sfr_model_registry(conn,
  conda_channel = "conda-forge",
  conda_channel_strict = TRUE
)
# All sfr_log_model() calls through `reg` now force conda-forge.
# Users cannot override the channel.

## End(Not run)
```

sfr_monitor_config *Model monitor registry configuration*

Description

Specifies the model version and warehouse used when registering a monitor via `sfr_add_monitor()`.

Usage

```
sfr_monitor_config(
  model_name,
  version_name,
  function_name = "predict",
  warehouse,
  aggregation_window = "1 day"
)
```

Arguments

`model_name`	Character. Registered model name.
`version_name`	Character. Model version name.
`function_name`	Character. Monitored prediction function name. Default: `"predict"`.
`warehouse`	Character. Warehouse for background monitor compute.
`aggregation_window`	Character. Aggregation window, e.g. `"1 day"`.

Value

An object of class `sfr_monitor_config`.

Examples

```
## Not run:
cfg <- sfr_monitor_config("MY_MODEL", "v1", warehouse = "ML_WH")

## End(Not run)
```

sfr_monitor_drift *Query drift metrics for a model monitor*

Description

Runs `SELECT * FROM TABLE(MODEL_MONITOR_DRIFT_METRIC(...))`.

Usage

```
sfr_monitor_drift(
  conn,
  monitor_name,
  metric,
  column,
  granularity = "DAY",
  start_time,
  end_time,
  segment = NULL
)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character. Model monitor name.
metric	Character. Drift metric (e.g. JENSEN_SHANNON).
column	Character. Feature, prediction, or actual column.
granularity	Character. Default "DAY" (mapped to '1 DAY'). See Snowflake documentation for other values.
start_time	Start timestamp (Date/POSIXct or SQL expression).
end_time	End timestamp (Date/POSIXct or SQL expression).
segment	Optional named list with column and value for segmented queries (JSON SEGMENTS passed as the last argument).

Value

A data.frame of results.

Examples

```
## Not run:
sfr_monitor_drift(
  conn, "MY_MON", "JENSEN_SHANNON", "SCORE",
  start_time = Sys.Date() - 30, end_time = Sys.Date()
)

## End(Not run)
```

---

sfr_monitor_performance

Query performance metrics for a model monitor

Description

Runs SELECT * FROM TABLE(MODEL_MONITOR_PERFORMANCE_METRIC(...)).

Usage

```
sfr_monitor_performance(
  conn,
  monitor_name,
  metric,
  granularity = "DAY",
  start_time,
  end_time,
  segment = NULL
)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.
metric	Character (e.g. RMSE, ROC_AUC).
granularity	Character. Default "DAY".
start_time, end_time	Timestamp bounds (Date/POSIXct or SQL expression).
segment	Optional list with column and value.

Value

A data.frame.

Examples

```
## Not run:
sfr_monitor_performance(
  conn, "MY_MON", "RMSE",
  start_time = Sys.Date() - 7, end_time = Sys.Date()
)

## End(Not run)
```

---

sfr_monitor_source *Model monitor source configuration*

Description

Describes the table (or view) and columns used as the monitor data source. Pass the result to sfr_add_monitor() as source_config.

Usage

```
sfr_monitor_source(
  source,
  timestamp_column,
  prediction_score_columns,
  actual_score_columns = NULL,
  id_columns = NULL
)
```

Arguments

source	Character. Table or view name (optionally fully qualified).
timestamp_column	Character. Timestamp column in the source.
prediction_score_columns	Character vector of prediction score columns.
actual_score_columns	Optional character vector of actual score columns.
id_columns	Optional character vector of ID columns (defaults to empty when passed to the Python registry).

Value

An object of class `sfr_monitor_source`.

Examples

```
## Not run:
src <- sfr_monitor_source(
  "MY_DB.MY_SCH.INFERENCE_LOG",
  "EVENT_TIME",
  prediction_score_columns = "PREDICTION"
)

## End(Not run)
```

---

sfr_monitor_stats *Query count / stat metrics for a model monitor*

Description

Runs `SELECT * FROM TABLE(MODEL_MONITOR_STAT_METRIC(...))` (Snowflake function name `MODEL_MONITOR_STAT_METRIC`).

Usage

```
sfr_monitor_stats(
  conn,
  monitor_name,
  metric,
  column,
  granularity = "DAY",
  start_time,
  end_time
)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.
metric	Character. COUNT or COUNT_NULL.
column	Character. Column to measure.
granularity	Character. Default "DAY".
start_time, end_time	Timestamp bounds.

Value

A data.frame.

Examples

```
## Not run:
sfr_monitor_stats(
  conn, "MY_MON", "COUNT", "SCORE",
  start_time = Sys.Date() - 7, end_time = Sys.Date()
)

## End(Not run)
```

---

sfr_monitor_to_vetiver

Fetch monitor metrics in a vetiver-like tibble

Description

Queries drift, performance, or stat metrics and returns a small tibble-class data frame with columns .index, .metric, and .estimate for plotting or tidymodels workflows.

Usage

```
sfr_monitor_to_vetiver(
  conn,
  monitor_name,
  metric,
  metric_type = "drift",
  start_time,
  end_time,
  column = NULL,
  granularity = "DAY"
)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.
metric	Character. Metric name for the chosen metric_type.
metric_type	One of "drift", "performance", or "stats".
start_time, end_time	Passed through to the underlying query function.
column	Required for drift and stats; ignored for performance.
granularity	Passed through.

Value

A data.frame with classes tbl_df, tbl, data.frame.

Examples

```
## Not run:
sfr_monitor_to_vetiver(
  conn, "MY_MON", "RMSE", metric_type = "performance",
  start_time = Sys.Date() - 7, end_time = Sys.Date()
)

## End(Not run)
```

---

sfr_online_config *Create an Online Config*

Description

Configuration object for enabling online feature serving on a Feature View. When passed to sfr_create_feature_view(or sfr_update_feature_view(), creates an Online Feature Table backed by a hybrid table.

Usage

```
sfr_online_config(enable = TRUE, target_lag = "1 minute")
```

Arguments

enable	Logical. Whether to enable online serving. Default: TRUE.
target_lag	Character. Target lag for the online table (e.g., "1 minute", "5 minutes"). The data in the online table will be at most this far behind the offline source.

Value

An sfr_online_config object.

Examples

```
## Not run:
online <- sfr_online_config(target_lag = "1 minute")

## End(Not run)
```

---

sfr_parcel_iterator *Create a parcel iterator for bulk I/O in doSnowflake*

Description

Groups a vector of partition keys (e.g. SKU IDs) into parcels of size parcel_size. Each parcel is a character vector that the foreach body can process as a batch, enabling a single stage read and write per parcel instead of per-SKU.

Usage

```
sfr_parcel_iterator(keys, parcel_size = 50L)
```

Arguments

keys	Character vector of partition keys.
parcel_size	Integer. Number of keys per parcel (default 50).

Value

A list of character vectors, one per parcel.

Examples

```
## Not run:
skus <- paste0("SKU_", 1:200)
parcels <- sfr_parcel_iterator(skus, parcel_size = 50)
# parcels is a list of 4 character vectors, each with 50 SKU IDs

library(foreach)
results <- foreach(parcel = parcels, .combine = rbind) %dopar% {
  # Bulk read data for all SKUs in parcel
  # Train models for each
  # Bulk write results
}
```

```
## End(Not run)
```

`sfr_pat` *Get a Snowflake authentication token for REST inference*

Description

Resolves an auth token from (in order):

1. The `token` argument (if provided)
2. The `SNOWFLAKE_PAT` environment variable
3. The `SNOWFLAKE_TOKEN` environment variable
4. The SPCS OAuth token at `/snowflake/session/token` (Workspace Notebooks)
5. A JWT generated from the connection's key-pair auth (if `conn` provided)

Usage

```
sfr_pat(token = NULL, conn = NULL)
```

Arguments

`token`	Character or NULL. An explicit token (PAT or JWT).
`conn`	An `sfr_connection` object, or NULL. When provided and no explicit token is available, generates a JWT from the session's key-pair credentials.

Value

Character. The token string.

Examples

```
## Not run:
Sys.setenv(SNOWFLAKE_PAT = "ver:1-hint:1234-ETMs...")
sfr_pat()

## End(Not run)
```

`sfr_predict` *Run remote inference with a registered model*

Description

Run remote inference with a registered model

Usage

```
sfr_predict(
  reg,
  model_name,
  new_data,
  version_name = NULL,
  service_name = NULL,
  partition_column = NULL,
  strict_input_validation = NULL,
  ...
)
```

Arguments

`reg`	An `sfr_model_registry` or `sfr_connection` object.
`model_name`	Character. Name of the registered model.
`new_data`	A data.frame with input data.
`version_name`	Character. Version to use (default: model's default).
`service_name`	Character. SPCS service name for container inference.
`partition_column`	Character or NULL. Column name to partition inference by (for large-scale batch prediction).
`strict_input_validation`	Logical or NULL. If `TRUE`, validates that input columns match the model's expected schema exactly.
`...`	Additional arguments.

Value

A data.frame with predictions.

`sfr_predict_body` *Convert an R function to a predict_body template*

Description

Takes a function with formals `(model, input)` and converts its body into the `{{MODEL}}`/`{{INPUT}}`/`{{UID}}` template string expected by `sfr_log_model()`.

Usage

```
sfr_predict_body(fn)
```

Arguments

`fn`	A function with exactly two formals. The first is treated as the model object; the second as the input data.frame. The function body must assign its final result to a variable named `result`.

Details

This lets you write and test your predict logic as a normal R function, then pass the template to `sfr_log_model(predict_body = ...)` without manually constructing `paste()` strings with placeholder syntax.

Value

Character scalar. The function body with formals replaced by `{{MODEL}}`, `{{INPUT}}`, and local variables suffixed with `{{UID}}`.

See Also

`sfr_log_model()`, `sfr_predict_local()`

Examples

```
## Not run:
my_predict <- function(model, input) {
  nd     <- as.matrix(input[, c("X1", "X2"), drop = FALSE])
  pred   <- predict(model, newdata = nd)
  result <- data.frame(prediction = as.numeric(pred))
}
body_str <- sfr_predict_body(my_predict)
mv <- sfr_log_model(reg, model = fit, model_name = "MY_MODEL",
                    predict_body = body_str, ...)

## End(Not run)
```

`sfr_predict_local` *Test an R model locally*

Description

Calls the R predict function directly on the model, exactly as the registered model would behave inside Snowflake. Use this to verify predictions before registering.

Usage

```
sfr_predict_local(
  model,
  new_data,
  predict_fn = "predict",
  predict_pkgs = character(0),
  predict_body = NULL,
  input_cols = NULL,
  output_cols = NULL
)
```

Arguments

`model`	An R model object.
`new_data`	A data.frame with input data.
`predict_fn`	Character. R function name for inference. Default: `"predict"`.
`predict_pkgs`	Character vector. R packages to load before prediction.
`predict_body`	Character. Optional custom R code. Use template variables `{{MODEL}}`, `{{INPUT}}`, `{{UID}}`, `{{N}}` (same as used in the Python bridge).
`input_cols`	Named list. Input column schema (for validation only).
`output_cols`	Named list. Output column schema (for validation only).

Details

Note: This runs entirely in R (no Python bridge). The Python bridge with rpy2 is only used when the model executes inside Snowflake (SPCS). Calling reticulate -> Python -> rpy2 -> R from within R would cause a dual-runtime segfault.

Value

A data.frame with predictions.

See Also

`sfr_log_model()`

Examples

```
## Not run:
model <- lm(mpg ~ wt, data = mtcars)
preds <- sfr_predict_local(model, data.frame(wt = c(2.5, 3.0, 3.5)))

## End(Not run)
```

`sfr_predict_rest` *Run model inference directly via the SPCS REST endpoint*

Description

Sends a data.frame to the model's HTTP endpoint and returns predictions. This is the fastest inference path – pure R, no Python bridge.

Usage

```
sfr_predict_rest(
  endpoint,
  new_data,
  token = NULL,
  conn = NULL,
  method = "predict",
  timeout_sec = 30
)
```

Arguments

`endpoint`	An `sfr_endpoint` object from `sfr_service_endpoint()`, or a character URL string.
`new_data`	A data.frame with input data.
`token`	Character or NULL. Auth token (PAT or JWT). If NULL, resolved via `sfr_pat()` (reads `SNOWFLAKE_PAT` env var, or auto-generates a JWT from `conn`).
`conn`	An `sfr_connection` object, or NULL. Used for auto-generating a JWT when no explicit token or PAT env var is available.
`method`	Character. Model method name. Default: `"predict"`. Underscores are converted to dashes in the URL per Snowflake convention.
`timeout_sec`	Numeric. Request timeout in seconds. Default: 30.

Value

A data.frame with predictions.

Examples

```
## Not run:
ep <- sfr_service_endpoint(conn, "mpg_service")
preds <- sfr_predict_rest(ep, new_data, conn = conn)

## End(Not run)
```

`sfr_predict_sql` *Run model inference via SQL (SQL-direct)*

Description

Invokes `model!function_name(*)` on the results of a SQL query. Bypasses the Python bridge entirely for inference, which is useful for large-scale batch scoring where the data is already in Snowflake.

Usage

```
sfr_predict_sql(
  conn,
  model_name,
  sql,
  version_name = NULL,
  function_name = "predict",
  parse_json = TRUE
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`model_name`	Character. Fully-qualified or unqualified model name.
`sql`	Character. SQL query whose results become the model input.
`version_name`	Character. Model version. If `NULL`, uses the default version.
`function_name`	Character. Model function to call. Default: `"predict"`.
`parse_json`	Logical. If `TRUE` (default), parse the JSON VARIANT output from `MODEL()!predict()` into a clean data.frame with numeric columns. Set to `FALSE` to return the raw JSON strings.

Value

A data.frame with prediction results.

Examples

```
## Not run:
conn <- sfr_connect()
preds <- sfr_predict_sql(
  conn,
  "ML_DB.MODELS.MY_MODEL",
  "SELECT col1, col2 FROM scoring_table",
  version_name = "v1"
)

## End(Not run)
```

`sfr_query` *Execute a SQL query and return results*

Description

Runs a SQL query against Snowflake and returns the result as an R data.frame. When an `RSnowflake` DBI connection is available on the `sfr_connection` (see `sfr_dbi_connection()`), the query is executed via the pure-R REST API path. Otherwise, it falls back to the Python Snowpark bridge.

Usage

```
sfr_query(
  conn,
  sql,
  .keep_case = !getOption("snowflakeR.lowercase_columns", FALSE)
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`sql`	Character. SQL query string.
`.keep_case`	Logical. If `TRUE` (default), preserve original column name casing from Snowflake. Set to `FALSE` to lowercase column names. The default respects the global option `snowflakeR.lowercase_columns`.

Details

Column names are returned **as-is from Snowflake** by default (UPPER case for unquoted identifiers). This matches the Python connector, Snowpark, and RSnowflake DBI behaviour and ensures consistency with the Model Registry / SPCS inference pipeline.

To globally restore the legacy lowercase behaviour, set `options(snowflakeR.lowercase_columns = TRUE)`.

Value

A data.frame with query results.

Examples

```
## Not run:
conn <- sfr_connect()
result <- sfr_query(conn, "SELECT CURRENT_TIMESTAMP() AS now")
result$NOW

## End(Not run)
```

sfr_queue_status *Get queue status for a running job*

Description

Returns pending, running, done, and failed counts plus throughput estimates for a queue-based job.

Usage

```
sfr_queue_status(conn, job_id, queue_fqn = "CONFIG.DOSNOWFLAKE_QUEUE")
```

Arguments

conn	An sfr_connection object.
job_id	Character. Job ID to check.
queue_fqn	Character. Queue table name.

Value

A list with total, pending, running, done, failed counts and throughput_per_min estimate.

sfr_read_dataset *Read a dataset version into an R data.frame*

Description

Reads the materialised stage files of a dataset version and returns them as a data.frame.

Usage

```
sfr_read_dataset(conn, name, version)
```

Arguments

conn	An sfr_connection object.
name	Character. Dataset name.
version	Character. Version to read.

Value

A data.frame.

sfr_read_feature_view *Read data from a Feature View*

Description

Read rows from the offline or online store backing a registered Feature View, optionally filtering by primary-key values and/or selecting specific feature columns.

Usage

```
sfr_read_feature_view(
  fs,
  name,
  version,
  store_type = "OFFLINE",
  keys = NULL,
  feature_names = NULL
)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to read.
store_type	Character. "OFFLINE" (default) or "ONLINE". Online reads require snowflake-ml-python >= 1.18. and the Feature View must have online serving enabled.
keys	A named list of primary-key columns and values to filter. Names must match the entity join_keys. For a single-key entity: list(CUSTOMER_ID = c(1L, 3L)). For composite keys: list(COL1 = c("a","b"), COL2 = c("x","y")). NULL (default) returns all rows.
feature_names	Character vector of feature column names to return. NULL (default) returns all features.

Value

A data.frame with feature data.

sfr_read_table *Read an entire table into a data.frame*

Description

Read an entire table into a data.frame

Usage

```
sfr_read_table(conn, table_name, limit = NULL)
```

Arguments

conn	An sfr_connection object.
table_name	Character. Name of the table.
limit	Integer. Maximum rows to return. Default: no limit.

Value

A data.frame.

---

sfr_refresh *Refresh connection context from the live session*

Description

Queries the Snowpark session for the current warehouse, database, schema, and role, and updates the cached fields on the connection object.

Usage

```
sfr_refresh(conn)
```

Arguments

conn	An sfr_connection object.

Details

This is rarely needed because $ access on an sfr_connection already queries the live session. Use this when you want to bulk-update the cached fields (e.g., before serialization).

Value

The updated sfr_connection object (invisibly). You **must** reassign: conn <- sfr_refresh(conn).

---

sfr_refresh_feature_view *Refresh a Feature View*

Description

Manually triggers a refresh of a managed Feature View.

Usage

```
sfr_refresh_feature_view(fs, name, version, store_type = "OFFLINE")
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to refresh.
store_type	Character. Store to refresh: "OFFLINE" (default) or "ONLINE".

Value

Invisibly returns TRUE.

---

sfr_register_feature_view

Register a draft Feature View in Snowflake

Description

Materialises a locally created Feature View in the Snowflake Feature Store. The Feature View's entities must already be registered.

Usage

```
sfr_register_feature_view(
  fs,
  feature_view,
  version,
  overwrite = FALSE,
  block = TRUE
)
```

Arguments

fs	An sfr_feature_store object.
feature_view	An sfr_feature_view object from sfr_feature_view().
version	Character. Version name (e.g., "v1").
overwrite	Logical. If TRUE, overwrites an existing version. Default: FALSE.
block	Logical. If TRUE (default), blocks until the feature view materialisation is complete.

Value

The sfr_feature_view object, updated with registration info.

See Also

sfr_feature_view(), sfr_create_feature_view()

sfr_reinstall *Reinstall and reload snowflakeR (and RSnowflake) from source*

Description

Convenience wrapper for the full reload sequence needed when developing inside a Workspace Notebook. Detaches loaded packages, reinstalls from source, reloads libraries, and refreshes all Python bridge modules.

Usage

```
sfr_reinstall(
  path = Sys.getenv("SNOWFLAKER_PATH"),
  rsnowflake_path = Sys.getenv("RSNOWFLAKE_PATH")
)
```

Arguments

path	Path to the snowflakeR source directory. Defaults to the SNOWFLAKER_PATH environment variable.
rsnowflake_path	Path to the RSnowflake source directory. Defaults to the RSNOWFLAKE_PATH environment variable. Set to "" to skip RSnowflake reinstallation.

Details

For most code changes (R function bodies, Python bridge files, NAMESPACE edits) this is sufficient. See the package dev standards (Section 13.5) for cases that require a kernel or container restart instead.

Value

Invisibly returns TRUE.

sfr_reload_bridges *Reload all cached Python bridge modules*

Description

Clears the R-side bridge cache and forces Python to re-import each module from disk on the next call. Useful during development when bridge code has changed without a kernel restart.

Usage

```
sfr_reload_bridges()
```

`sfr_result_scan` *Retrieve a previous query's results via RESULT_SCAN*

Description

Executes `SELECT * FROM TABLE(RESULT_SCAN('<query_id>'))` against Snowflake. The query ID can be provided explicitly, or looked up from the notebook query tracker by cell execution count or `dataframe_N` name.

Usage

```
sfr_result_scan(
  conn,
  query_id = NULL,
  cell = NULL,
  dataframe = NULL,
  .keep_case = !getOption("snowflakeR.lowercase_columns", FALSE)
)
```

Arguments

`conn`	An `sfr_connection` object from `sfr_connect()`.
`query_id`	Character. An explicit Snowflake query ID. If `NULL`, looks up the ID via `cell` or `dataframe`.
`cell`	Integer. The IPython cell execution count (`In[N]`).
`dataframe`	Character. The Workspace `dataframe_N` variable name.
`.keep_case`	Logical. If `TRUE` (default), preserve original column name casing from Snowflake.

Details

Requires the query tracker to be installed (automatic when using `setup_notebook()` in a Workspace Notebook).

Value

A data.frame with the query results.

Examples

```
## Not run:
conn <- sfr_connect()

# Explicit query ID
df <- sfr_result_scan(conn, query_id = "01c34f1a-0814-fa4a-0000-0c09644ace8a")

# By dataframe name (from SQL cell)
df <- sfr_result_scan(conn, dataframe = "dataframe_4")

# By cell number
df <- sfr_result_scan(conn, cell = 3)
```

```
# Most recent user query
df <- sfr_result_scan(conn)

## End(Not run)
```

`sfr_resume_compute_pool` *Resume a compute pool*

Description

Resume a compute pool

Usage

```
sfr_resume_compute_pool(conn, name)
```

Arguments

`conn`	An `sfr_connection` object.
`name`	Character. Compute pool name.

Value

Invisibly returns `TRUE`.

`sfr_resume_feature_view` *Resume a Feature View*

Description

Resumes automatic refresh of a suspended Feature View.

Usage

```
sfr_resume_feature_view(fs, name, version)
```

Arguments

`fs`	An `sfr_feature_store` object.
`name`	Character. Feature View name.
`version`	Character. Version to resume.

Value

Invisibly returns `TRUE`.

sfr_resume_monitor *Resume a model monitor*

Description

Resume a model monitor

Usage

```
sfr_resume_monitor(conn, monitor_name)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.

Value

Invisibly TRUE.

Examples

```
## Not run:
sfr_resume_monitor(conn, "MY_MON")

## End(Not run)
```

sfr_retrieve_features *Retrieve feature values for inference*

Description

Calls fs.retrieve_feature_values() in the Python SDK. Without spine_timestamp_col this performs a simple LEFT JOIN on entity keys and returns the **current** feature values. When spine_timestamp_col is provided the join is point-in-time correct (same logic as sfr_generate_training_data()).

Usage

```
sfr_retrieve_features(
  fs,
  spine,
  features,
  spine_timestamp_col = NULL,
  exclude_columns = NULL,
  include_feature_view_timestamp_col = FALSE,
  auto_prefix = FALSE,
  join_method = NULL
)
```

Arguments

fs	An sfr_feature_store object.
spine	A data.frame, SQL string, or dbplyr lazy table with entity key values.
features	A list of registered sfr_feature_view objects or lists with name and version.
spine_timestamp_col	Optional timestamp column in the spine for point-in-time feature value lookup. When NULL (the default) the latest feature values are returned.
exclude_columns	Character vector or NULL. Column names to exclude from the result.
include_feature_view_timestamp_col	Logical. If TRUE, includes the timestamp column from each feature view. Default: FALSE.
auto_prefix	Logical. If TRUE, prefixes feature columns with the feature view name to avoid collisions. Default: FALSE.
join_method	Character or NULL. Join strategy (e.g. "AS_OF", "INNER"). When NULL, the SDK picks a sensible default.

Value

A data.frame with feature values.

---

sfr_run_batch *Run batch inference via SPCS*

Description

Run batch inference via SPCS

Usage

```
sfr_run_batch(
  reg,
  model_name,
  version_name,
  new_data,
  compute_pool = NULL,
  function_name = "predict"
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
new_data	A data.frame with input data.
compute_pool	Character. Compute pool for the batch job.
function_name	Character. Function to call (default: "predict").

Value

A data.frame with predictions.

`sfr_run_executor` *Run a Registered R Executor*

Description

Convenience wrapper that builds and executes the SQL to call a registered R executor model as a TABLE_FUNCTION.

Usage

```
sfr_run_executor(
  conn,
  model_name,
  input_sql,
  partition_by,
  version_name = NULL,
  function_name = "execute"
)
```

Arguments

`conn`	An `sfr_connection` object.
`model_name`	Character. Registered model name.
`input_sql`	Character. SQL query providing input data. Must include the partition column(s).
`partition_by`	Character. Column name for `PARTITION BY`.
`version_name`	Character. Version to use. If `NULL`, uses default.
`function_name`	Character. Method name (default `"execute"`).

Value

A data.frame with the executor output.

Examples

```
## Not run:
conn <- sfr_connect()
results <- sfr_run_executor(
  conn,
  model_name = "SKU_OPTIMISER",
  input_sql = "SELECT SKU, PARAMS FROM WORK_MANIFEST",
  partition_by = "SKU"
)
print(results)

## End(Not run)
```

sfr_run_executor_batch *Run Executor with run_batch (SPCS Compute Pool)*

Description

Uses Model Registry's `run_batch` method to execute the registered R executor on a specified compute pool with automatic scaling.

Usage

```
sfr_run_executor_batch(
  reg,
  model_name,
  version_name,
  input_data,
  compute_pool,
  function_name = "execute"
)
```

Arguments

`reg`	An `sfr_model_registry` or `sfr_connection` object.
`model_name`	Character. Registered model name.
`version_name`	Character. Version label.
`input_data`	Data frame or SQL query string providing input data.
`compute_pool`	Character. SPCS compute pool name.
`function_name`	Character. Method name (default `"execute"`).

Value

A data.frame with executor output.

sfr_scale_worker_pool *Scale a persistent worker pool*

Description

Alters the MIN/MAX instances of an existing SPCS service to scale the worker pool up or down.

Usage

```
sfr_scale_worker_pool(conn, service_name, replicas)
```

Arguments

`conn`	An `sfr_connection` object.
`service_name`	Character. Name of the SPCS service.
`replicas`	Integer. New number of replicas.

Value

Invisibly returns the service name.

`sfr_service_endpoint` *Discover the REST endpoint URL for an SPCS model service*

Description

Queries Snowflake for the endpoint URL of a deployed SPCS service. The returned object can be passed directly to `sfr_predict_rest()`.

Usage

```
sfr_service_endpoint(conn, service_name, database = NULL, schema = NULL)
```

Arguments

`conn`	An `sfr_connection` object.
`service_name`	Character. Name of the SPCS service.
`database`	Character or NULL. Override the database for service lookup.
`schema`	Character or NULL. Override the schema for service lookup.

Value

An `sfr_endpoint` object containing the URL and metadata.

Examples

```
## Not run:
ep <- sfr_service_endpoint(conn, "mpg_service")
ep
# <sfr_endpoint>
# url:     https://xyz-myorg123.snowflakecomputing.app
# service: MYDB.MYSCHEMA.MPG_SERVICE

## End(Not run)
```

`sfr_setup_feature_store`
Set up a Feature Store schema

Description

Provisions the required schema, roles, and privileges for a Feature Store. This is typically a one-time admin operation.

Usage

```
sfr_setup_feature_store(conn, database, schema, warehouse)
```

Arguments

conn	An sfr_connection object.
database	Character. Database for the Feature Store.
schema	Character. Schema for the Feature Store.
warehouse	Character. Default warehouse.

Value

Invisibly returns TRUE.

---

sfr_set_default_model_version *Set the default version of a model*

Description

Set the default version of a model

Usage

```
sfr_set_default_model_version(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

Invisibly returns TRUE.

---

sfr_set_model_alias *Set a model version alias (SQL-direct)*

Description

Assigns a named alias (e.g., "production", "staging") to a model version. Aliases provide stable references that can be updated without changing downstream consumers.

Usage

```
sfr_set_model_alias(conn, model_name, version_name, alias)
```

Arguments

conn	An sfr_connection object from sfr_connect().
model_name	Character. Fully-qualified or unqualified model name.
version_name	Character. Version to alias.
alias	Character. Alias name (e.g., "production").

Value

Invisibly returns TRUE.

---

sfr_set_model_metric *Set a metric on a model version*

Description

Set a metric on a model version

Usage

```
sfr_set_model_metric(reg, model_name, version_name, metric_name, metric_value)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
metric_name	Character. Name of the metric.
metric_value	Numeric or character. Value of the metric.

Value

Invisibly returns TRUE.

---

sfr_show_fv_versions *Show versions of a Feature View*

Description

Show versions of a Feature View

Usage

```
sfr_show_fv_versions(fs, name)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.

Value

A data.frame with version information.

---

sfr_show_models *List models in the registry*

Description

List models in the registry

Usage

```
sfr_show_models(reg)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.

Value

A data.frame listing registered models.

---

sfr_show_model_functions *List callable functions on a model version*

Description

List callable functions on a model version

Usage

```
sfr_show_model_functions(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

A data.frame listing functions.

sfr_show_model_metrics

Show metrics for a model version

Description

Show metrics for a model version

Usage

```
sfr_show_model_metrics(reg, model_name, version_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.

Value

A named list of metrics.

sfr_show_model_monitors

List model monitors in the registry

Description

List model monitors in the registry

Usage

```
sfr_show_model_monitors(reg)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.

Value

A data.frame of monitors (columns depend on the Python SDK).

Examples

```
## Not run:
sfr_show_model_monitors(reg)

## End(Not run)
```

sfr_show_model_versions *Show versions of a model*

Description

Show versions of a model

Usage

```
sfr_show_model_versions(reg, model_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Name of the model.

Value

A data.frame with version information.

sfr_slice_feature_view *Select a subset of features from a Feature View*

Description

Select a subset of features from a Feature View

Usage

```
sfr_slice_feature_view(fs, name, version, feature_names)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version.
feature_names	Character vector of feature names to keep.

Value

An sfr_feature_view object with only the selected features.

sfr_snowflake_version *Get Snowflake server version*

Description

Get Snowflake server version

Usage

```
sfr_snowflake_version(conn)
```

Arguments

conn	An sfr_connection object from sfr_connect().

Value

A character string with the Snowflake server version.

Examples

```
## Not run:
conn <- sfr_connect()
sfr_snowflake_version(conn)
#> [1] "9.26.0"

## End(Not run)
```

sfr_start_run *Start an experiment run*

Description

Start an experiment run

Usage

```
sfr_start_run(exp, name = NULL)
```

Arguments

exp	An sfr_experiment object from sfr_experiment().
name	Character. Optional run name.

Value

Invisibly TRUE.

sfr_status *Check connection status*

Description

Check connection status

Usage

```
sfr_status(conn)
```

Arguments

conn	An sfr_connection object.

Value

Invisibly returns TRUE if the connection is active.

sfr_stop_worker_pool *Stop and remove a worker pool*

Description

Stop and remove a worker pool

Usage

```
sfr_stop_worker_pool(conn, service_name)
```

Arguments

conn	An sfr_connection object.
service_name	Character. Name of the SPCS service.

Value

Invisibly returns TRUE.

sfr_storage_config *Create an Iceberg StorageConfig*

Description

Configuration for Iceberg-backed Feature Views. Requires `snowflake-ml-python >= 1.26.0`.

Usage

```
sfr_storage_config(
  format = "ICEBERG",
  external_volume = NULL,
  base_location = NULL
)
```

Arguments

`format`	Character. Storage format. Currently only `"ICEBERG"`.
`external_volume`	Character. External volume name.
`base_location`	Character. Base location path within the volume.

Value

An `sfr_storage_config` object.

sfr_suspend_compute_pool *Suspend a compute pool*

Description

Suspend a compute pool

Usage

```
sfr_suspend_compute_pool(conn, name)
```

Arguments

`conn`	An `sfr_connection` object.
`name`	Character. Compute pool name.

Value

Invisibly returns `TRUE`.

sfr_suspend_feature_view *Suspend a Feature View*

Description

Pauses automatic refresh of a managed Feature View.

Usage

```
sfr_suspend_feature_view(fs, name, version)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Feature View name.
version	Character. Version to suspend.

Value

Invisibly returns TRUE.

sfr_suspend_monitor *Suspend a model monitor*

Description

Suspend a model monitor

Usage

```
sfr_suspend_monitor(conn, monitor_name)
```

Arguments

conn	An sfr_connection object.
monitor_name	Character.

Value

Invisibly TRUE.

Examples

```
## Not run:
sfr_suspend_monitor(conn, "MY_MON")

## End(Not run)
```

sfr_table_exists *Check if a table exists*

Description

Check if a table exists

Usage

```
sfr_table_exists(conn, table_name)
```

Arguments

conn	An sfr_connection object.
table_name	Character. Name of the table (may be fully qualified as DB.SCHEMA.TABLE).

Value

Logical.

sfr_undeploy_model *Undeploy a model service*

Description

Undeploy a model service

Usage

```
sfr_undeploy_model(reg, model_name, version_name, service_name)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character.
version_name	Character.
service_name	Character.

Value

Invisibly returns TRUE.

sfr_unset_model_alias *Unset a model alias (SQL-direct)*

Description

Removes an alias from a model version. The underlying version remains but is no longer reachable via the alias name. The version is referenced by its alias in the SQL command.

Usage

```
sfr_unset_model_alias(conn, model_name, alias)
```

Arguments

conn	An sfr_connection object from sfr_connect().
model_name	Character. Fully-qualified or unqualified model name.
alias	Character. Alias name to remove.

Value

Invisibly returns TRUE.

sfr_update_default_warehouse *Update the default warehouse for the Feature Store*

Description

Update the default warehouse for the Feature Store

Usage

```
sfr_update_default_warehouse(fs, warehouse_name)
```

Arguments

fs	An sfr_feature_store object.
warehouse_name	Character. New default warehouse name.

Value

Invisibly returns TRUE.

sfr_update_entity *Update an entity's description*

Description

Update an entity's description

Usage

```
sfr_update_entity(fs, name, desc)
```

Arguments

fs	An sfr_feature_store object.
name	Character. Entity name.
desc	Character. New description.

Value

Invisibly returns TRUE.

sfr_update_executor *Update R Executor Code*

Description

Registers a new version of an existing R executor with updated R code. This is the recommended way to update R logic without changing the model infrastructure.

Usage

```
sfr_update_executor(
  reg,
  model_name,
  r_script_path,
  version_name = NULL,
  set_default = TRUE,
  ...
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Existing model name.
r_script_path	Character. Path to the updated R script.
version_name	Character. New version label. If NULL, auto-generated.
set_default	Logical. If TRUE (default), sets the new version as the default.
...	Additional arguments passed to sfr_log_executor().

Value

A list with `model_name`, `version_name`, `registration_time_s`.

Examples

```
## Not run:
# Update the R code and set as default
sfr_update_executor(
  conn,
  model_name = "SKU_OPTIMISER",
  r_script_path = "optimise_sku_v2.R",
  set_default = TRUE
)

## End(Not run)
```

`sfr_update_feature_view` *Update a Feature View*

Description

Updates properties of an existing registered Feature View.

Usage

```
sfr_update_feature_view(
  fs,
  name,
  version,
  refresh_freq = NULL,
  warehouse = NULL,
  desc = NULL,
  online_config = NULL
)
```

Arguments

`fs`	An `sfr_feature_store` object.
`name`	Character. Feature View name.
`version`	Character. Version to update.
`refresh_freq`	Character. New refresh frequency (optional).
`warehouse`	Character. New warehouse (optional).
`desc`	Character. New description (optional).
`online_config`	An `sfr_online_config()` object (optional).

Value

Invisibly returns `TRUE`.

sfr_upload_r_script *Upload an R Script to a Snowflake Stage*

Description

Uploads a local R script to a Snowflake stage, ready for execution by the generic R executor. The script must define an entry function (default `main(config)`) that the executor will call.

Usage

```
sfr_upload_r_script(
  conn,
  script_path,
  stage = "DOSNOWFLAKE_STAGE",
  stage_dir = "scripts"
)
```

Arguments

conn	An `sfr_connection` object from `sfr_connect()`.
script_path	Character. Local path to the R script.
stage	Character. Stage name (default `"DOSNOWFLAKE_STAGE"`).
stage_dir	Character. Directory within the stage (default `"scripts"`).

Value

The stage-relative path for use in `sfr_execute_r_script()`, e.g. `"/stage/scripts/my_script.R"`.

Examples

```
## Not run:
conn <- sfr_connect()
stage_path <- sfr_upload_r_script(conn, "my_analysis.R")
# stage_path == "/stage/scripts/my_analysis.R"

## End(Not run)
```

sfr_use *Switch warehouse, database, or schema*

Description

Runs `USE WAREHOUSE/DATABASE/SCHEMA` on the Snowpark session and updates the connection object. **Important:** R objects are pass-by-value, so you must reassign the result: `conn <- sfr_use(conn, schema = "NEW")`.

Usage

```
sfr_use(conn, warehouse = NULL, database = NULL, schema = NULL)
```

Arguments

conn	An sfr_connection object.
warehouse	Character. New warehouse name.
database	Character. New database name.
schema	Character. New schema name.

Value

The updated sfr_connection object (invisibly). You **must** reassign: conn <- sfr_use(conn, ...).

---

sfr_vetiver_to_metrics

Store vetiver metrics as model registry metrics

Description

Takes output from vetiver_compute_metrics() (vetiver package) and stores each metric as a model version metric via sfr_set_model_metric().

Usage

```
sfr_vetiver_to_metrics(
  reg,
  model_name,
  version_name,
  vetiver_metrics,
  prefix = "vetiver_"
)
```

Arguments

reg	An sfr_model_registry or sfr_connection object.
model_name	Character. Model name.
version_name	Character. Version name.
vetiver_metrics	A data.frame from vetiver_compute_metrics() with columns .index, .metric, .estimator, .estimate.
prefix	Character. Prefix for metric names to distinguish vetiver metrics from other metrics. Default: "vetiver_".

Value

Invisibly returns the number of metrics stored.

`sfr_wait_for_service` *Wait for an SPCS service to be ready*

Description

Polls `sfr_get_service_status()` until the service reaches `READY/RUNNING` or a terminal failure state is detected. Useful after `sfr_deploy_model()` since SPCS services take several minutes to provision.

Usage

```
sfr_wait_for_service(
  reg,
  service_name,
  timeout_min = 10,
  poll_sec = 15,
  verbose = TRUE
)
```

Arguments

`reg`	An `sfr_model_registry` or `sfr_connection` object. Used to resolve the database and schema where the service lives.
`service_name`	Character. Name of the SPCS service.
`timeout_min`	Numeric. Maximum minutes to wait. Default: 10.
`poll_sec`	Numeric. Seconds between status checks. Default: 15.
`verbose`	Logical. Print status updates? Default: `TRUE`.

Value

`TRUE` (invisibly) if the service is running; raises an error on terminal failure or timeout.

Examples

```
## Not run:
sfr_deploy_model(reg, "MY_MODEL", "V1", "my_svc", "ML_POOL", "my_repo")
sfr_wait_for_service(reg, "my_svc", timeout_min = 15)

## End(Not run)
```

sfr_worker_pool_status *Get the status of a worker pool*

Description

Get the status of a worker pool

Usage

```
sfr_worker_pool_status(conn, service_name)
```

Arguments

conn	An sfr_connection object.
service_name	Character. Name of the SPCS service.

Value

A list with name, status, and instance count.

sfr_workspace_setup *Set up the R environment in a Snowflake Workspace Notebook*

Description

One-call setup that configures the R environment, installs ADBC drivers, and sets up PAT management for Workspace Notebooks.

Usage

```
sfr_workspace_setup(install_adbc = TRUE)
```

Arguments

install_adbc	Logical. Whether to install ADBC drivers. Default: TRUE.

Value

Invisibly returns TRUE.

`sfr_write_table` *Write a data.frame to a Snowflake table*

Description

By default, column names are uppercased before writing so that the resulting Snowflake columns are stored as standard unquoted identifiers (e.g. `CYL`, not `"cyl"`).

Usage

```
sfr_write_table(
  conn,
  table_name,
  value,
  overwrite = FALSE,
  .uppercase_cols = !getOption("snowflakeR.preserve_write_case", FALSE)
)
```

Arguments

`conn`	An `sfr_connection` object.
`table_name`	Character. Name of the target table.
`value`	A data.frame to write.
`overwrite`	Logical. If `TRUE`, replaces the table. If `FALSE` (default), appends or creates.
`.uppercase_cols`	Logical. If `TRUE` (default), column names are uppercased before writing. Set to `FALSE` to preserve original casing. Default respects `getOption("snowflakeR.preserve_write_cas`

Details

This is necessary because the Snowpark `create_dataframe(pandas_df)` path double-quotes **every** column name from the pandas DataFrame to preserve its exact case. Without uppercasing first, an R data.frame with lowercase names like `mtcars` would produce quoted lowercase columns (`"mpg"`, `"cyl"`) that are unreachable via normal unquoted SQL. The uppercasing is applied to a **local copy** – the caller's original data.frame is not modified (R copy-on-modify semantics).

Set `.uppercase_cols = FALSE` to preserve the original column names exactly (they will be stored as quoted identifiers in Snowflake and must be referenced with double-quotes in SQL).

Value

Invisibly returns `TRUE`.

stopDoSnowflake *Stop the cached doSnowflake cluster*

Description

Shuts down the reusable socket cluster that `registerDoSnowflake()` maintains across `\%dopar\%` calls.

Usage

```
stopDoSnowflake()
```

Value

Invisibly returns `NULL`.

$.sfr_connection *Access elements of an sfr_connection*

Description

Intercepts access to session context fields (`warehouse`, `database`, `schema`, `role`) and queries the live Snowpark session so the values always reflect the current state – including changes made via the Snowsight UI picker, SQL cells, or Python cells.

Usage

```
## S3 method for class 'sfr_connection'
x$name
```

Arguments

`x`	An `sfr_connection` object.
`name`	Element name.

Value

The element value.

Index